



PENINSULA
COLLEGE



UNIVERSITY OF
PLYMOUTH

MAL3018

COMPUTING PROJECT

AI-Based HR Resume Recommender System

Prepared by,

Muhammad Iman Haqimi Bin Zaidi

BSSE 2409165

BSC (HONS) Computer Science

(Software Engineering)

Year 3 Semester 2

Acknowledgements

I would like to express my sincere gratitude to **Sir Eric Kong Kok Wah** for his guidance, feedback, and continuous support throughout this project. I also extend my appreciation to **Mrs Tracy**, the HR executive from **PKT** who participated in the stakeholder interview, for sharing valuable real-world insights that directly shaped the system requirements and design decisions. Finally, I would like to thank everyone who contributed advice, encouragement, and assistance during the development, testing, and completion of this project.

Abstract

This project presents an AI-Based HR Resume Recommender System designed to reduce the time, inconsistency, and bias risks associated with manual resume screening. In conventional recruitment workflows, HR personnel often process large volumes of applications under severe time constraints, leading to subjective and inconsistent evaluation and limited transparency for candidates. To address this, a full-stack web application was developed to automate resume analysis and generate ranked candidate lists using an interpretable AI scoring approach. The solution is implemented as a three-tier microservice architecture. The frontend is built with HTML, CSS, and vanilla JavaScript, providing separate portals for HR personnel and job candidates. The backend is a RESTful API developed with Node.js and Express.js, connected to MongoDB for data persistence and secured with JWT-based role-based access control. A dedicated Python FastAPI microservice performs Natural Language Processing (NLP) on submitted resumes, extracting education and skills using spaCy pattern matching and computing relevance using TF-IDF. Candidate-job fit is calculated using a deterministic weighted formula (Education 40% + Skills 60%), enabling consistent and explainable scoring. The system supports end-to-end recruitment screening workflows: job posting, resume upload (PDF/DOCX/TXT), application submission, AI scoring, ranked candidate views for HR, status updates (shortlist/reject), and CSV export. Candidates can track application status and view match scores, improving transparency. User Acceptance Testing confirmed that core user flows function correctly and that the platform provides fast, structured ranking and a practical dashboard to support HR decision-making.

Table of Contents

Acknowledgements	1
Abstract.....	1
Table of Contents	2
Word Count	7
Code Repository	7
Link To YouTube	7
Link to PebblePad	7
Chapter 1: Introduction	8
1.1 Project Overview.....	8
1.2 Problem Statement	9
1.3 Project Aim	10
1.4 Project Objectives.....	10
1.5 Scope of the Project.....	11
1.6 Significance of the Project	12
1.7 Project Contributions	13
1.8 Structure of the Report	14
Chapter 2: Background, Objectives and Deliverables.....	15
2.1 Background	15
2.1.1 Context of the Problem	15
2.1.2 Motivation	16
2.1.3 Existing Solutions and Systems	17
2.1.4 Literature Review	19
2.2 Objectives	23
2.3 Deliverables.....	25
2.3.1 Internal Deliverables.....	25
2.3.2 Final Deliverables	26
Chapter 3: Method of Approach	27
3.1 Risk Assessment.....	27
3.1.1 Risk Register.....	27
3.1.2 Risk Matrix.....	29
3.1.3 Risk Monitoring.....	29
3.2 Requirements Analysis.....	30
3.2.2 Non-Functional Requirements	32

3.2.3 User Stories.....	33
3.2.4 Minimum Viable Product (MVP) Definition	33
3.3 Technologies Used	34
3.3.1 Frontend	34
3.3.2 Backend.....	35
3.3.3 AI Module	36
3.4 Best Practices.....	37
3.4.1 Code Structure and Modularity	37
3.5 Testing Strategy	38
3.5.1 Unit Testing	38
3.5.2 Integration Testing	38
3.5.3 User Acceptance Testing (UAT)	39
3.5.4 Manual vs Automated Testing.....	39
3.6 System Design	40
3.6.1 Architecture Diagram.....	40
3.6.2 Data Flow — Candidate Ranking Process.....	41
3.6.3 Activity Diagram	42
3.6.4 Database Design (NoSQL — MongoDB)	43
3.7 Assumptions and Constraints	44
3.7.3 Project Limitations.....	44
Chapter 4: Legal, Social and Professional Issues	45
4.1 Social Issues	45
4.1.1 User Interaction Implications	45
4.1.2 Fairness and Algorithmic Bias	45
4.1.3 Usability and Accessibility	46
4.1.4 Impact on Different User Groups.....	47
4.2 Legal Issues.....	48
4.2.1 Data Protection — Malaysia PDPA 2010.....	48
4.2.2 International Reference — GDPR.....	49
4.2.3 Privacy Considerations	49
4.2.4 Intellectual Property	50
4.3 Professional Issues	51
4.3.1 Ethical Guidelines — ACM and BCS.....	51
4.3.2 Security Considerations.....	52
4.3.3 Responsible Software Development.....	53
4.3.4 API Usage and Licensing	54

Chapter 5: Project Management	55
5.1 Project Management Tools	55
5.1.1 Tools Used	55
5.1.2 Task Tracking and Prioritisation	56
5.2 Methodology.....	57
5.2.1 Choice of Methodology — Agile (Scrum)	57
5.2.2 Sprint Planning	57
5.2.3 Project Gantt Chart.....	58
5.2.4 Backlog Management	59
5.3 Monitoring and Control	59
5.3.1 Sprint Reviews.....	59
5.3.3 Time Management and Workload Distribution.....	61
Chapter 6: Project Stages / Development Phases	62
6.1 Stage 1 — Planning and Preparation (Weeks 1–5).....	63
6.1.1 Problem Analysis.....	63
6.1.2 Literature Review	63
6.1.3 Initial Design and Proposal.....	63
6.2 Stage 2 — Design and Setup (Weeks 5–9).....	64
6.2.1 System Architecture	64
6.2.2 UI Mockups and Interface Design	64
6.2.3 Environment Setup	66
6.3 Stage 3 — Implementation (Weeks 9–15).....	67
6.3.1 Backend Development (Node.js / Express.js)	67
6.3.2 AI Module Development (Python / FastAPI / spaCy)	68
6.3.3 Frontend Development (HTML / CSS / JavaScript)	69
6.4 Stage 4 — Testing and Refinement (Weeks 15–18).....	71
6.4.1 Bug Fixing.....	71
6.4.2 Performance Improvements.....	71
6.4.3 User Testing Feedback	72
6.5 Stage 5 — Finalisation (Weeks 18–20).....	72
6.5.1 Final Testing	72
6.5.2 Documentation	72
6.5.3 Report Writing	72
6.6 Stage Overview — Reflection on Progress	73
6.6.1 Progress Across Stages	73
6.6.2 Lessons Learned	73

Chapter 7: Output Documentation	74
7.1 Output Diagrams	74
7.1.1 Final System Architecture Diagram.....	74
7.1.2 Final UML Diagrams.....	75
7.1.3 Comparison with Initial Design.....	77
7.2 Database Structure.....	79
7.2.1 Final Database Design — MongoDB Collection Schema	79
7.2.2 Design Rationale	80
7.3 Completed Requirements / User Stories	81
7.3.1 MVP Features — Implemented	81
7.3.2 Post-MVP Features — Implemented	82
7.3.3 Confirmation of Project Objectives.....	83
7.4 User Testing Results	84
7.4.1 Description of User Testing Conducted	84
7.4.2 User Acceptance Testing Results.....	84
7.4.3 Key Usability Findings and Observations.....	85
7.5 Final Interface Output Screenshots	86
Chapter 8: End-Project Report / Evaluation	90
8.1 Design Adherence.....	90
8.1.1 Proposed vs Final Implemented Design	90
8.1.2 Architecture Changes	92
8.1.3 UI/UX Changes	92
8.1.4 Technology Changes.....	92
8.2 Objective Achievement	93
8.3 System Evaluation	95
8.3.1 Strengths of the System	95
8.3.2 Limitations and Weaknesses.....	96
8.3.3 Performance, Usability, and Reliability.....	97
8.4 Future Enhancements.....	98
Chapter 9: Post-mortem / Critical Reflection.....	99
9.1 Technology Review.....	99
9.2 Project Management Reflection.....	101
9.3 Personal Performance.....	102
9.4 User Testing Reflection.....	103
9.5 Ethical and Accessibility Reflection	103
9.6 Overall Reflection	104

Chapter 10: Conclusion	105
10.1 Project Summary	105
10.1.1 Recap of the Problem Addressed.....	105
10.1.2 Summary of the Developed Solution.....	105
10.2 Achievement of Aim and Objectives.....	106
10.2.1 Confirmation of How the Project Aim Was Met	106
10.2.2 High-Level Summary of Objective Completion	106
10.3 Contribution of the Project	107
10.3.1 Academic Contribution.....	107
10.3.2 Practical and Industry Relevance	107
10.3.3 Learning Outcomes Achieved.....	107
10.4 Final Remarks	108
10.4.1 Overall Success Statement	108
10.4.2 Readiness for Real-World Deployment	108
10.4.3 Closing Reflection	108
References	109
Appendix	111
Appendix A — User Interview Transcript (PKT HR Stakeholder Interview)	111
Appendix B — API Endpoint Reference	114
Appendix C — Skill Library	118

Word Count

Word Count: 10,925

Code Repository

Link to GitHub: <https://github.com/ImanHaqimi/hr-resume-recommender>

Link To YouTube

https://youtu.be/4GvZ_RpG9Hw

Link to PebblePad

<https://v3.pebblepad.co.uk/spa/#/public/mtw79Zrj5kfmpRgykG5qgtgmzw>

Chapter 1: Introduction

1.1 Project Overview

The **AI-Based HR Resume Recommender System** is a full-stack web application designed to assist Human Resource (HR) professionals in efficiently screening and ranking job candidates using Artificial Intelligence (AI) and Natural Language Processing (NLP). The system automates the resume analysis process by comparing candidate resumes against job requirements, producing ranked and scored candidate lists for HR review.

The system serves two primary groups of users:

- **HR Personnel** – who create and manage job postings, view AI-ranked candidate lists, manage application statuses, export candidate data, and make informed hiring decisions.
- **Job Candidates** – who register on the platform, build their profiles, upload their resumes, browse available job listings, apply for positions, and track the status of their applications in real time.

The platform is built using a three-tier architecture:

- A **frontend** built with HTML5, CSS3, and vanilla JavaScript, providing separate portals for HR and candidates.
- A **backend** RESTful API built with Node.js and Express.js, connected to a MongoDB database.
- A **Python AI microservice** built with FastAPI, leveraging NLP techniques including pattern-based skill extraction and semantic matching using spaCy and TF-IDF (Term Frequency-Inverse Document Frequency) — a proven text representation technique established by Salton and Buckley (1988) — to analyse resumes and rank candidates.

Development follows an **Agile (Scrum)** methodology with iterative testing and refinement throughout the project lifecycle.

1.2 Problem Statement

What Problem Exists?

The traditional recruitment process is highly manual, time-consuming, and prone to human bias. HR departments in organisations of all sizes routinely receive large volumes of job applications for each vacancy. According to LinkedIn Talent Solutions (2025), recruiters spend an average of only six to seven seconds reviewing each resume during initial screening — a rate that makes thorough, consistent evaluation practically impossible when handling hundreds of applications for a single role.

Key issues in conventional recruitment include:

- **Volume overload:** HR personnel struggle to review all submitted resumes thoroughly, especially for popular job postings.
- **Inconsistency:** Human screening is subjective — different recruiters may evaluate the same resume with different criteria and outcomes.
- **Time-to-hire:** The slow manual screening process leads to delayed hiring decisions, which can result in losing top talent to competitors.
- **Skill mismatch:** Without structured analysis, relevant candidate qualifications may be overlooked or mismatched against job requirements.
- **Lack of transparency:** Candidates rarely receive clear feedback on how well their profile matches the job requirements.

Why Is It Important to Solve?

In today's competitive labour market, efficient recruitment is critical to an organisation's success. Delays in hiring suitable candidates can directly impact productivity and business operations. In Malaysia, widely used recruitment platforms such as JobStreet (JobStreet Malaysia, 2025) rely predominantly on keyword-based filtering, which lacks the semantic understanding needed to accurately match candidate qualifications to job requirements. Furthermore, Bogen and Rieke (2018) highlight that keyword-based filtering systems commonly used in Applicant Tracking Systems (ATS) can inadvertently exclude qualified candidates whose resumes do not precisely match predefined terms, a form of automated bias that disadvantages applicants without reducing the screening effort for HR. Implementing an AI-powered screening solution addresses these challenges by providing faster, more consistent, and data-driven

candidate evaluations. Candidates also benefit from a more transparent application process where they can view their match scores and application statuses in real time.

1.3 Project Aim

The overall aim of this project is to **design, develop, and deploy an AI-powered web-based HR resume recommender system** that automates the candidate screening and ranking process, reducing the time and effort required by HR professionals to identify the most suitable candidates for a given job role, while supporting candidates in understanding how well their profiles match available positions.

1.4 Project Objectives

The following objectives were defined for this project:

1. **To design and develop a web-based system** that allows HR personnel to post job vacancies and candidates to submit their resumes through a user-friendly interface.
2. **To implement an AI model** that analyses and ranks candidate resumes based on education level and skills matching the job description and automatically filters out unqualified applicants.
3. **To provide HR with a simple dashboard** that displays ranked candidates and presents clear match scores to support informed hiring decisions.

1.5 Scope of the Project

In Scope

- HR can post, edit, and manage job vacancies through a dedicated HR portal.
- HR can view AI-ranked candidates per job, update application statuses (Pending, Shortlisted, Reviewed, Rejected), and export candidate shortlists to CSV.
- HR dashboard provides real-time statistics through summary cards, a donut chart visualising Shortlisted vs Rejected outcomes, and horizontal status bars for at-a-glance screening progress.
- Candidates can register, create/update their profile, upload resumes (PDF, DOCX, TXT), apply for jobs, and track application status (Pending, Shortlisted, Rejected) with AI match scores.
- The backend system handles resume uploads, stores all data in MongoDB, and communicates with the AI module.
- The AI module analyses candidate education levels and skills, ranks suitable candidates, and automatically filters out unqualified applicants.
- Role-based access control with JWT authentication for HR and Candidate accounts.

Out of Scope

- Resume builders or resume editing tools.
- Interview scheduling and chat/communication features.
- Integration with third-party job boards or HR platforms.
- Mobile application development (iOS/Android).

1.6 Significance of the Project

This project holds significance across multiple dimensions:

Benefits to Users

- **HR professionals** save considerable time and effort by receiving an automatically ranked, scored list of candidates instead of manually screening each resume individually.
- **Candidates** benefit from a transparent system where they can view their match scores and understand how well their qualifications align with each job, enabling them to make better application decisions.

Benefits to Industry

- The system provides a cost-effective, scalable recruitment screening solution suitable for small-to-medium enterprises (SMEs) and startups that do not have the budget for enterprise-grade HR software.
- Its domain-independent design means it can be deployed across a wide variety of industries without modification or retraining of the AI model.

Benefits to Academia

- The project demonstrates the practical application of AI and NLP techniques in a real-world Human Resource Management (HRM) context, contributing to research on automated talent acquisition systems.
- The microservice architecture adopted in this project reflects current industry best practices in full-stack development and serves as an educational reference for the integration of AI capabilities into web applications.

1.7 Project Contributions

This project makes the following specific and measurable contributions to the problem of manual resume screening:

Problem 1: Time-Consuming Manual Screening

Solved by: The AI scoring module automates the evaluation of every submitted resume against the job's required skills and education level. HR personnel no longer need to read each resume individually — they are immediately presented with a ranked, scored list of candidates. This reduces initial screening time from hours to seconds per job posting.

Problem 2: Inconsistency and Subjectivity in Evaluation

Solved by: A deterministic, rule-based scoring algorithm (Education 40% + Skills 60%) is applied uniformly to every candidate for the same job. Unlike human screening, the system evaluates the same resume the same way every time, eliminating the inconsistency caused by varying reviewer standards, attention, or unconscious bias.

Problem 3: Lack of Transparency for Candidates

Solved by: The system surfaces each candidate's AI match score and application status (Pending, Shortlisted, Rejected) directly on their portal. Unlike conventional platforms such as JobStreet or Indeed, where candidates receive no feedback on their application outcome, this system provides a clear, data-driven explanation of how well their qualifications match the role.

Problem 4: Difficulty Managing Multiple Job Postings

Solved by: The manage-jobs page allows HR to create, view, edit, and delete multiple job postings in one centralised interface. Each posting includes configurable required skills, education thresholds, and a match score filter, enabling structured management of concurrent recruitment campaigns.

Problem 5: No Structured Candidate Shortlisting Workflow

Solved by: The system provides a complete end-to-end shortlisting workflow — from AI-ranked candidate lists to one-click status updates (Shortlist / Reject), to CSV export of shortlisted candidates for sharing with hiring managers — replacing ad-hoc manual processes with a structured, auditable workflow.

1.8 Structure of the Report

The report is organised as follows:

- **Chapter 1 – Introduction:** Introduces the project, outlines the problem statement, defines the project aim and objectives, and describes the scope and significance of the system.
- **Chapter 2 – Background, Objectives and Deliverables:** Reviews existing literature and related systems in the field of AI-powered recruitment. Outlines the project's objectives and the deliverables produced throughout the development lifecycle.
- **Chapter 3 – Method of Approach:** Describes the software development methodology adopted for this project, including the rationale for the chosen approach and the tools and technologies used.
- **Chapter 4 – Legal, Social and Professional Issues:** Examines the ethical, legal, and professional considerations relevant to the development and deployment of an AI-based recruitment system, including data privacy and algorithmic fairness.
- **Chapter 5 – Project Management:** Presents the project planning artefacts, including the work breakdown structure, Gantt chart, risk register, and resource allocation plan.
- **Chapter 6 – Project Stages / Development Phases:** Documents the iterative development process, covering the design, implementation, and testing of each major system component across all development phases.
- **Chapter 7 – Output Documentation:** Provides technical documentation for the system, including system architecture diagrams, database schema, API documentation, and UI screenshots.
- **Chapter 8 – End-Project Report / Evaluation:** Evaluates the final system against the defined objectives and requirements, including the results of functional and non-functional testing.
- **Chapter 9 – Post-mortem / Critical Reflection:** Reflects critically on the development experience, highlighting lessons learned, challenges encountered, and recommendations for future improvements.
- **Chapter 10 – Conclusion:** Summarises the project's outcomes, revisits the original aims and objectives, and presents closing remarks.

Chapter 2: Background, Objectives and Deliverables

2.1 Background

2.1.1 Context of the Problem

Recruitment is one of the most critical and resource-intensive functions within any organisation. In today's competitive labour market, companies often receive hundreds of applications for a single job opening, making manual resume screening both time-consuming and cognitively demanding for HR personnel.

According to LinkedIn Talent Solutions (2025), the volume of job applications per posting has increased significantly in recent years, with recruiters spending an average of six to seven seconds reviewing each resume during initial screening. This rapid, high-volume format increases the likelihood of qualified candidates being overlooked due to cognitive fatigue, inconsistent evaluation criteria, and the inherent limitations of keyword-based filtering.

In Malaysia, popular recruitment platforms such as JobStreet are widely used by both employers and job seekers (JobStreet Malaysia, 2025). While these platforms serve as effective aggregators of job applications, their built-in screening tools remain largely keyword-dependent, lacking deeper semantic understanding of how a candidate's qualifications truly relate to a job role. This gap creates inefficiencies on both ends: HR personnel spend excessive time manually reviewing unqualified applications, while suitable candidates may be filtered out simply because their resumes do not contain the exact keywords expected by the system.

The growing availability of AI and Natural Language Processing (NLP) technologies offers a viable solution to these challenges. The use of TF-IDF (Term Frequency-Inverse Document Frequency) for text representation was pioneered by Salton and Buckley (1988) and remains a widely used baseline for document similarity tasks. More recently, distributed word representations such as Word2Vec (Mikolov et al., 2013) and transformer-based models such as BERT (Devlin et al., 2019) have demonstrated significant improvements in semantic text understanding, enabling more meaningful comparisons between candidate qualifications and job requirements. However, most implementations of these advanced technologies in recruitment remain embedded within large enterprise platforms that are inaccessible to small-to-medium enterprises (SMEs) and startups.

2.1.2 Motivation

This project was motivated by the practical recognition that recruitment inefficiency is a problem faced by organisations of all sizes, yet accessible intelligent screening tools remain limited. The motivation behind the AI-Based HR Resume Recommender System stems from three key observations:

Accessibility gap — Advanced AI-powered recruitment tools are predominantly embedded within large, costly enterprise platforms. Smaller organisations and academic institutions lack access to equivalent intelligent screening capabilities.

Skill-qualification mismatch — Traditional keyword-based filtering fails to capture the semantic relationship between a candidate's actual skills and the competencies required by a job. This leads to both false positives (unqualified candidates passing the filter) and false negatives (qualified candidates being incorrectly rejected).

Academic relevance — The intersection of AI, NLP, and Human Resource Management (HRM) represents a growing area of research and practical application. Building a working system in this domain provides a valuable demonstration of how modern AI techniques can be applied to real-world workflow problems.

The project therefore aims to fill this gap by developing a lightweight, domain-independent, web-based screening system that is affordable, accessible, and effective for organisations that do not have enterprise-grade HR platforms.

2.1.3 Existing Solutions and Systems

Several existing platforms and systems address aspects of automated resume screening. The table below provides a brief comparison of notable existing solutions against the proposed system:

Table 2.1: Comparison of Existing Recruitment Platforms and the Proposed System

Feature	LinkedIn	JobStreet	ATS (e.g., Workday)	Proposed System
Resume Parsing	✓	✓	✓	✓
AI-Based Ranking	Partial	✗	Partial	✓
Match Score Display	✗	✗	✗	✓
Candidate Self-Tracking	✓	✓	Partial	✓
Configurable Threshold	✗	✗	✗	✓
Domain Independent	Limited	Limited	Limited	✓
Accessible to SMEs	Limited	✓	✗ (costly)	✓

LinkedIn is one of the most widely used professional networking and recruitment platforms globally. While it provides recruiter-facing filtering tools, its AI-driven matching features are largely restricted to premium-tier subscriptions (LinkedIn Talent Solutions, 2025). Furthermore, matching quality is heavily dependent on how thoroughly candidates have populated their profiles, creating an inherent bias against candidates who are less digitally active, regardless of their actual qualifications.

JobStreet Malaysia is the dominant job portal in the Malaysian market and is commonly used by both fresh graduates and experienced professionals (JobStreet Malaysia, 2025). However, its candidate filtering remains keyword-based. There is no built-in AI ranking system — HR users must manually sort through applicant lists without any automated relevance scoring, which negates the efficiency gain that technology should provide.

Applicant Tracking Systems (ATS) such as Workday, SAP SuccessFactors, and Taleo are widely adopted by large enterprises and provide sophisticated HR workflow automation. While these systems include resume parsing and candidate databasing, they are commonly criticised for over-reliance on rigid keyword filtering, which can result in qualified candidates being automatically rejected if their resumes do not precisely match predefined criteria (Bogen & Rieke, 2018). Additionally, the high implementation and licensing costs make these tools inaccessible to SMEs.

The proposed system addresses the core limitations of all three categories: it provides transparent AI-generated match scores visible to both HR and candidates, supports configurable filtering thresholds per job posting, and operates independently of industry domain — all within a lightweight, open-source web application accessible without premium subscriptions or enterprise licensing.

2.1.4 Literature Review

2.1.4.1 Artificial Intelligence in Recruitment

The application of AI in recruitment has grown significantly over the past decade. Cappelli (2019) notes that AI-driven tools improve screening speed but introduce challenges around transparency and fairness when systems learn from biased historical data. Upadhyay and Khandelwal (2018) concluded that AI reduces time-to-hire but is most effective when combined with human decision-making. This informed the design decision to retain HR control over all final shortlisting decisions, with the AI acting as a ranking aid. Lindsey et al. (2013) found that structured, criteria-based assessment significantly reduces unconscious bias — reflected in this project's fixed scoring formula (Education 40%, Skills 60%) applied consistently to every candidate.

2.1.4.2 Natural Language Processing for Resume Parsing

Maheshwari et al. (2010) proposed an NER and rule-based information extraction framework for unstructured resume text — the approach directly applied in this project via spaCy. Liao et al. (2018) found that pattern-based extraction remains competitive for English-language professional documents, validating the choice of spaCy's PhraseMatcher over more complex deep learning models given project constraints. Qin et al. (2018) demonstrated that education level, work experience, and skills collectively contribute to job-resume compatibility, informing the scoring weights adopted in this system (Education 40%, Skills 60%).

2.1.4.3 TF-IDF and Text Similarity in Recruitment

Salton and Buckley (1988) established TF-IDF as a foundational document similarity method that remains widely used for short, structured professional texts. Zhang et al. (2016) found TF-IDF outperformed keyword frequency counting and was significantly faster than neural embedding approaches, making it appropriate for real-time candidate ranking. More recent transformer-based models — BERT (Devlin et al., 2019) and Sentence-BERT (Reimers & Gurevych, 2019) — offer improved semantic similarity and are recommended as future enhancements.

2.1.4.4 Deep Learning vs. Rule-Based Approaches

Ferreira et al. (2022) found that for domain-specific structured documents such as resumes, rule-based extraction pipelines remain highly competitive due to their determinism, interpretability, and lower computational cost. This project prioritises an explainable scoring model over statistical superiority — appropriate given that scoring decisions directly affect hiring outcomes that must be justifiable to both candidates and HR.

2.1.4.5 Ethical Considerations in AI Hiring Tools

Bogen and Rieke (2018) identified that keyword-based filtering disproportionately disadvantages candidates from minority or non-traditional backgrounds. Raghavan et al. (2020) recommended that hiring AI systems document scoring criteria, provide candidate feedback, and retain human decision-makers. This project responds directly: the scoring formula is fully documented (Section 3.3), match scores and statuses are visible to candidates (FR-C6), and all shortlisting remains with HR (Section 4.1).

2.1.4.6 Resume Format and Parsing Challenges

Harsha et al. (2020) identified that image-based PDFs present significant parsing challenges requiring OCR as a preprocessing step. This project uses PyPDF2 and pdfminer for text-based PDFs plus python-docx for DOCX files; image-based PDFs are acknowledged as a known limitation (Section 3.7), with OCR recommended as a future enhancement.

2.1.5 User and Stakeholder Input

To inform the design and functional requirements of the system, a structured interview was conducted with Mrs Tracy, an HR executive at PKT, a Malaysian-based organisation. The interview covered the organisation's current recruitment workflow, pain points, and expectations for an AI-based screening system.

Current Recruitment Process

Mrs Tracy explained that PKT currently posts job vacancies across multiple free platforms including Indeed, Facebook (company page), and MyFutureJobs, with JobStreet used occasionally for hard-to-fill roles. The preference for free platforms is driven by the organisation's zero hiring cost policy. Candidates submit applications primarily through the Indeed portal, though email and WhatsApp submissions are also accepted. PDF is the preferred resume format.

Screening Process

The current screening process follows a fixed priority order: education level is reviewed first, followed by location, then work experience. There is currently no formal evaluation template — decisions are made through manual resume review and informal background checks. Shortlisted candidates are then forwarded to the Hiring Manager, Department Head, and finally HQ for approval, a process that can take up to one week.

Pain Points Identified

The interview revealed several significant pain points in the current process:

Manual resume filtering is time-consuming, especially when handling large volumes of applications.

The multi-level approval process (HR → Hiring Manager → Department Head → HQ) creates bottlenecks and delays hiring decisions.

Difficulty finding skilled candidates was highlighted as an ongoing challenge, particularly for specialised roles.

Mrs Tracy confirmed that HR would welcome automation of the resume filtering and ranking stage to reduce time spent on initial screening.

System Expectations

When asked about the features expected from a new system, Mrs Tracy outlined the following requirements:

Automated candidate ranking based on job requirements.

A dashboard displaying candidate details including name, contact information, address, qualifications, and job history.

Visibility of candidate match scores, matched skills, education level, and a resume preview directly on the dashboard.

The ability to export candidate lists for sharing with the Hiring Manager and Department Head.

Mrs Tracy also noted that integration with existing platforms such as Indeed and JobStreet would be desirable.

Regarding data privacy, while no strict internal policy exists, she acknowledged that secure data storage is good practice.

Design Implications

The findings from this interview directly shaped several design decisions in the proposed system:

Table 2.2: Design Implications Derived from User Interview with Mrs Tracy (PKT)

Interview Finding	System Design Decision
Education reviewed first in screening	AI scoring weights education at 40% as primary criterion
HR wants visible match scores	Match scores displayed on both HR ranking page and candidate applications page
No formal evaluation template	Dashboard provides structured candidate view with score breakdown
PDF preferred format	System accepts PDF as primary format alongside DOCX and TXT
Future: skill gap analysis & scheduling	Noted as future enhancement in project scope

2.2 Objectives

The objectives of this project are defined using the SMART framework (Specific, Measurable, Achievable, Relevant, Time-bound) and are directly linked to the problem statement identified in Chapter 1.

Objective 1 — Web-Based System for HR and Candidates

Specific: Design and develop a web-based application with separate portals for HR personnel and job candidates, enabling HR to post job vacancies and candidates to submit resumes.

Measurable: The system must include functioning HR and candidate portals with login, job posting, resume upload, and application management features, verified through functional testing.

Achievable: Developed using well-established technologies: HTML/CSS/JavaScript for the frontend, Node.js/Express.js for the backend, and MongoDB as the database.

Relevant: Directly addresses the problem of HR inefficiency by creating a structured digital environment for job applications and candidate management.

Time-bound: Completed by the final development phase within the 20-week project timeline.

Objective 2 — AI Model for Resume Analysis and Ranking

Specific: Implement an AI module that parses submitted resumes, extracts relevant skills and education information using NLP techniques (spaCy, TF-IDF, pattern matching), and produces a ranked candidate list based on match scores.

Measurable: The AI module must successfully parse resumes in PDF, DOCX, and TXT formats, compute match scores (0–100%), and rank candidates in descending order of match. Performance verified via sample test scenarios with HR and candidate test accounts.

Achievable: Implemented as a Python FastAPI microservice using established NLP libraries and a weighted scoring algorithm (Education: 40%, Skills: 60%).

Relevant: Directly addresses the core problem — the inability of manual or keyword-based screening to accurately assess candidate suitability.

Time-bound: AI module fully integrated and tested within the development and testing phases of the project timeline.

Objective 3 — HR Dashboard with Ranked Candidates and Management Tools

Specific: Provide HR personnel with an interactive dashboard displaying real-time statistics, a ranked candidate list per job posting, match scores, filtering controls, job management capabilities, dual-chart status visualisation (horizontal status bars and a Shortlisted vs Rejected donut chart), and the ability to export candidate shortlists to CSV.

Measurable: The dashboard must display accurate candidate rankings and match scores, allow HR to update application statuses (Pending, Shortlisted, Reviewed, Rejected), manage active/closed job postings, render status visualisations correctly, and export candidate data. Candidates must also be able to view their application status (Pending, Shortlisted, or Rejected) alongside their AI match scores. All features verified through user acceptance testing.

Achievable: Implemented using the frontend HR portal (dashboard, candidate ranking, and manage-jobs pages) connected to the backend API, with real-time data refresh, status management, Chart.js for visualisations, and CSV export functionality.

Relevant: Directly supports HR in reducing time-to-hire by focusing attention on pre-ranked, qualified candidates and providing tools to manage the full recruitment workflow digitally. Candidates benefit from transparency through visible match scores and application outcomes.

Time-bound: All dashboard features fully functional and tested by the final verification phase of the project.

2.3 Deliverables

2.3.1 Internal Deliverables

Internal deliverables are the planning, documentation, and design artefacts produced throughout the development lifecycle of the project:

Table 2.3: Internal Deliverables

#	Deliverable	Description
1	Project Proposal	Initial proposal document outlining objectives, scope, methodology, and risk plan
2	System Architecture Diagram	Diagram illustrating the three-tier architecture (Frontend, Backend, AI Module)
3	Database Schema Design	MongoDB collection design for Users, Jobs, and Applications
4	UI Wireframes / Mockups	Design mockups for HR and Candidate portal pages
5	Gantt Chart	20-week project timeline with task phases and milestones
6	Risk Register	Identified risks, likelihood, impact, and mitigation strategies
7	API Documentation	Documentation of all backend REST API endpoints
8	Testing Report	Results of unit, integration, and AI model evaluation tests

2.3.2 Final Deliverables

The final deliverables represent the completed, working outputs of the project:

Table 2.4: Final Deliverables

#	Deliverable	Description
1	Web Application	Fully functional AI-Based HR Resume Recommender System accessible via web browser, with HR and Candidate portals
2	Backend API	Node.js/Express.js RESTful API server connected to MongoDB, handling authentication, job management, candidate ranking, status updates, and CSV export
3	AI Microservice	Python FastAPI service performing resume parsing, skill extraction, and candidate-job match scoring (Education 40% + Skills 60%)
4	HR Portal	Multi-page interface including dashboard (with summary cards, status bar charts, and donut chart), job posting, manage jobs, and AI-ranked candidate views with CSV export
5	Candidate Portal	Multi-page interface including registration, profile management, job listings, resume upload, job application, and application tracking (Pending / Shortlisted / Rejected) with AI match scores
6	Source Code Repository	Complete, version-controlled source code hosted on GitHub: https://github.com/ImanHaqimi/hr-resume-recommender
7	Final Report	Comprehensive project report covering all chapters from Introduction to Conclusion
8	Final Presentation	Presentation slides summarising the project, demonstrating the system, and presenting evaluation results

Chapter 3: Method of Approach

This chapter documents the planning, design, and implementation approach for the AI-Based HR Resume Recommender System, covering risk assessment, requirements analysis, technology selection, best practices, testing strategy, system design, and project constraints.

3.1 Risk Assessment

Risk assessment was conducted at project outset to identify threats to successful delivery. Each risk was evaluated for likelihood and impact, with a mitigation strategy defined.

3.1.1 Risk Register

Table 3.1: Risk Register

#	Risk	Category	Likelihood	Impact	Mitigation Strategy
R1	Difficulty integrating the Python AI microservice with the Node.js backend	Technical	Medium	High	Define a clear REST API contract between the two services early; test integration at the end of each sprint
R2	Resume parsing errors for non-standard PDF formats	Technical	Medium	High	Use multiple parsing libraries (PyPDF2, python-docx) as fallbacks; test with varied resume formats
R3	Insufficient time to complete all planned features within the 20-week timeline	Time	Medium	High	Use Agile sprint planning with a prioritised backlog; define a Minimum Viable Product (MVP) as the baseline
R4	Learning curve for new technologies (FastAPI, spaCy, Chart.js)	Learning	High	Medium	Allocate dedicated time in early sprints for technology familiarisation; use official documentation and community resources
R5	AI scoring accuracy being insufficient for	Technical	Low	High	Validate scoring with sample resumes against known expected outputs; tune

	meaningful candidate ranking				weighting (Education 40%, Skills 60%) iteratively
R6	Data security vulnerabilities (unauthorised access, token leakage)	Security	Low	High	Implement JWT authentication with role-based access control; hash all passwords using bcrypt
R7	Scope creep adding unplanned features and delaying delivery	Management	Medium	Medium	Strictly adhere to the defined scope in the project proposal; defer non-MVP features to the future enhancements list

3.1.2 Risk Matrix

To provide a visual overview of the severity distribution of identified risks, the risks from the register above are plotted on a Likelihood vs Impact matrix in Figure 3.1 below. Risks appearing in the red zone (High Impact, High Likelihood) require the most urgent mitigation, while those in the green zone are monitored but pose minimal threat to project delivery.

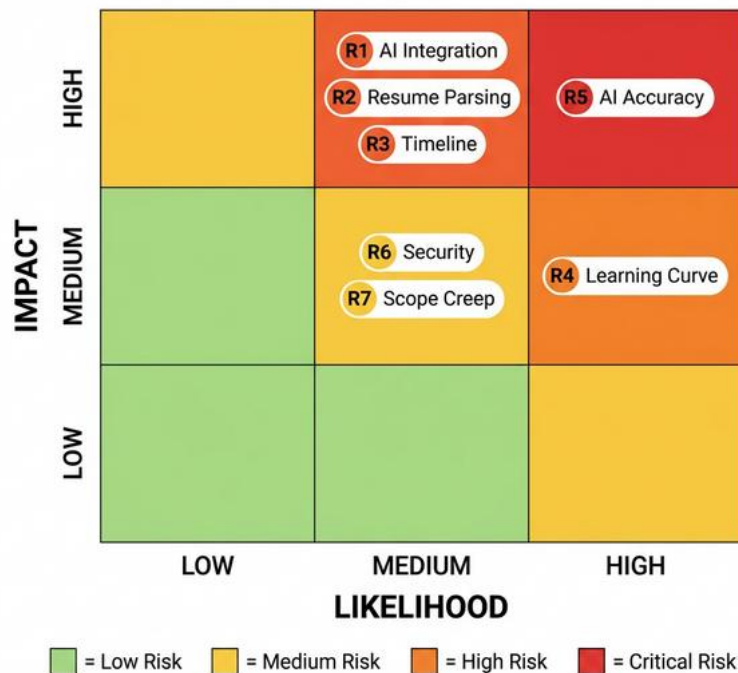


Figure 3.1: Risk Matrix — Likelihood vs Impact Heat Map

As shown in the matrix, the most critical risks are R1 (AI integration), R2 (resume parsing), and R3 (timeline) — all positioned in the High Impact zone. R4 (learning curve) is the highest likelihood risk but with only Medium impact, making it manageable through early sprint allocation.

3.1.3 Risk Monitoring

Throughout the project, risks were reviewed at the start of each sprint. Risks R1 and R4 materialised during early development — the AI-backend integration required additional API debugging, and initial learning time was needed for spaCy's pipeline and FastAPI's asynchronous request handling. Both were resolved within their respective sprints without impacting the overall project timeline.

3.2 Requirements Analysis

Requirements were gathered through a structured stakeholder interview (Mrs Tracy, PKT), review of existing platforms, and academic literature.

3.2.1 Functional Requirements

Functional requirements define the specific behaviours and feature the system must provide:

HR Portal:

Table 3.2: HR Portal Functional Requirements

ID	Requirement
FR-H1	HR users must be able to register and log in with email and password
FR-H2	HR users must be able to create, view, edit, and delete job postings
FR-H3	HR users must be able to view AI-ranked candidates for each job, sorted by match score (descending)
FR-H4	HR users must be able to update each application's status (Shortlisted, Reviewed, Rejected)
FR-H5	HR users must be able to filter the candidate list to show only qualified applicants (match score $\geq$ threshold)
FR-H6	HR users must be able to export the candidate shortlist for a job to a CSV file
FR-H7	The HR dashboard must display summary statistics: total applications, active jobs, shortlisted and rejected counts
FR-H8	The HR dashboard must display a donut chart (Shortlisted vs Rejected) and horizontal status bars

Candidate Portal:**Table 3.3: Candidate Portal Functional Requirements**

ID	Requirement
FR-C1	Candidates must be able to register and log in with email and password
FR-C2	Candidates must be able to upload their resume in PDF, DOCX, or TXT format
FR-C3	Candidates must be able to create and update their personal profile
FR-C4	Candidates must be able to browse available job listings
FR-C5	Candidates must be able to apply for a job using their uploaded resume
FR-C6	Candidates must be able to view their application status (Pending, Shortlisted, Rejected) and AI match score per application

System / AI:**Table 3.4: System and AI Module Functional Requirements**

ID	Requirement
FR-S1	The system must extract skills and education level from submitted resumes using NLP
FR-S2	The system must compute a match score (0–100%) for each candidate relative to each job requirement
FR-S3	The system must rank candidates in descending order of match score
FR-S4	The system must enforce role-based access — HR users and candidates can only access their own data

3.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes the system must meet:

Table 3.5: Non-Functional Requirements

Category	Requirement
Performance	The HR dashboard must load statistics and candidate rankings within 3 seconds under normal load
Security	All API routes must be protected by JWT authentication; passwords must be hashed using bcrypt; user data must not be accessible across roles
Usability	The interface must be intuitive for non-technical HR users; key actions (post job, view rankings, update status) must be accessible within two clicks
Scalability	The system must follow a microservice architecture so the AI module can be scaled or upgraded independently of the backend
Reliability	The AI module must handle parsing failures gracefully and return a meaningful error response without crashing the backend
Maintainability	Code must be modular, with clear separation between routes, controllers, models, middleware, and the AI service

3.2.3 User Stories

User stories were defined to capture requirements from the perspective of each user type:

- *As an HR user, I want to post a job with a required skills list and education level so that candidates are evaluated against specific criteria.*
- *As an HR user, I want to see candidates ranked by AI match score so that I can focus on the most suitable applicants first.*
- *As an HR user, I want to export the shortlisted candidates to CSV so that I can share the list with the Hiring Manager.*
- *As a candidate, I want to upload my resume and apply for jobs so that HR can evaluate my qualifications.*
- *As a candidate, I want to see my match score and application status so that I understand how well my profile fits the role.*

3.2.4 Minimum Viable Product (MVP) Definition

To manage scope and ensure timely delivery, an MVP was defined at the start of the project. The MVP required the following to be in place before any enhancements:

1. HR and Candidate registration and login with JWT authentication
2. Job posting with required skills and education level
3. Resume upload (PDF/DOCX)
4. AI analysis producing a match score (0–100%)
5. Ranked candidate list visible to HR
6. Application status update (Shortlist / Reject)
7. Candidate application tracking with match score display

All MVP features were implemented by Sprint 3, with additional features (CSV export, manage-jobs page, status bar charts, candidate profile) added in Sprints 4 and 5.

3.3 Technologies Used

Technology selection balanced performance, developer familiarity, community support, and suitability for the problem domain.

3.3.1 Frontend

Table 3.6: Frontend Technologies and Justifications

Technology	Justification
HTML5 / CSS3	Standard web technologies providing full control over structure and styling without additional build tooling
Vanilla JavaScript	Chosen over React or Vue.js to keep the frontend lightweight and avoid unnecessary framework overhead for a two-portal system with clearly defined page boundaries
Chart.js	A lightweight, Canvas-based charting library used to render the dashboard donut chart and status visualisations. Suitable for dynamic data from the backend API
Fetch API	Used for all HTTP requests to the backend; native browser support eliminates the need for additional HTTP client libraries

3.3.2 Backend

Table 3.7: Backend Technologies and Justifications

Technology	Justification
Node.js	Event-driven, non-blocking I/O runtime suitable for handling multiple concurrent API requests from both HR and candidate portals
Express.js	Minimal and flexible web framework for Node.js; used to define RESTful API routes with clear separation into auth, HR, and candidate route groups
MongoDB	Document-based NoSQL database well-suited to storing semi-structured data such as resumes, job postings, and applications where schema flexibility is advantageous
Mongoose	ODM (Object Data Modelling) library for MongoDB; enforces schema validation for User, Job, and Application collections
Multer	Node.js middleware for handling multipart/form-data file uploads; used for resume file ingestion
JSON Web Tokens (JWT)	Stateless authentication mechanism: tokens are issued on login and validated on each protected API route, supporting scalable and sessionless authentication
bcrypt	Industry-standard password hashing library; ensures plaintext passwords are never stored in the database

3.3.3 AI Module

Table 3.8: AI Module Technologies and Justifications

Technology	Justification
Python	The language of choice for NLP and AI tasks due to its rich ecosystem of data science and machine learning libraries
FastAPI	High-performance asynchronous Python web framework; chosen over Flask for its built-in data validation (Pydantic), automatic API documentation (Swagger UI), and async support
spaCy	Industrial-strength NLP library used for text preprocessing and pattern-based skill extraction from resume text
TF-IDF (Term Frequency-Inverse Document Frequency)	A proven statistical technique (Salton & Buckley, 1988) used to measure keyword relevance between resume content and job requirement descriptions
PyPDF2 / pdfminer	Python libraries for extracting text from PDF resumes — the preferred format identified through the PKT stakeholder interview
python-docx	Library for parsing DOCX resume submissions

3.4 Best Practices

3.4.1 Code Structure and Modularity

The backend follows a **layered architecture** with clear separation between routes, controllers, models, middleware, and services. Each frontend page has its own JavaScript file, with shared config centralised in

config.js. The AI module follows a pipeline pattern: parsers/ → analyzers/ → scoring functions.

Git was used throughout with the repository hosted at ImanHaqimi/hr-resume-recommender, using a feature-branch workflow (feature branches merged to main after local testing). All backend routes follow RESTful conventions; sensitive configuration is stored in

.env files excluded from version control. The AI microservice is fully decoupled from the backend and can be upgraded independently.

3.5 Testing Strategy

Testing was conducted across three levels to validate individual components and the integrated system.

3.5.1 Unit Testing

Individual components of the AI module were tested in isolation:

- The resume parser was tested with sample PDF, DOCX, and TXT files to verify correct text extraction
- The skill extractor was tested with known resume inputs to verify accurate detection of skills, education level, and years of experience
- The scoring function was tested with controlled inputs (e.g., a resume with all required skills should score close to 100%) to verify the weighting algorithm (Education 40%, Skills 60%)

3.5.2 Integration Testing

Integration testing verified end-to-end data flow across the three tiers:

- **Frontend → Backend:** Verified that API calls for login, job posting, and resume upload returned correct responses
- **Backend → AI Module:** Verified that submitted resumes were correctly forwarded to the FastAPI service and that ranked results were returned and stored
- **Backend → Database:** Verified that job postings, user accounts, and application records correctly persisted in MongoDB

3.5.3 User Acceptance Testing (UAT)

UAT was conducted using dedicated HR and candidate test accounts to simulate real-world usage:

Table 3.9: User Acceptance Testing Results

Test Case	Role	Expected Outcome	Result
Register and log in	Candidate	JWT token issued, redirect to portal	 Pass
Upload PDF resume	Candidate	Resume stored, confirmation shown	 Pass
Post job with skills	HR	Job saved, visible in manage-jobs	 Pass
View AI-ranked candidates	HR	Candidates ranked by score (descending)	 Pass
Update application to Shortlisted	HR	Status updated, reflected on candidate's page	 Pass
Export candidate list to CSV	HR	CSV file downloaded with correct data	 Pass
Filter applications by status	Candidate	Only matching status applications shown	 Pass

3.5.4 Manual vs Automated Testing

The primary testing approach was **manual testing** conducted through the browser interface and direct API calls (using browser developer tools). Automated test scripts were not implemented within the project timeline; this is noted as a future improvement. All AI module endpoints were also tested directly via the FastAPI auto-generated Swagger UI (<http://localhost:8000/docs>).

3.6 System Design

3.6.1 Architecture Diagram

The system follows a **three-tier microservice architecture**, as illustrated in Figure 3.2 below. The three layers — Frontend, Backend, and AI Module — communicate via HTTP REST APIs and operate on separate ports, enabling each tier to be developed, tested, and scaled independently.

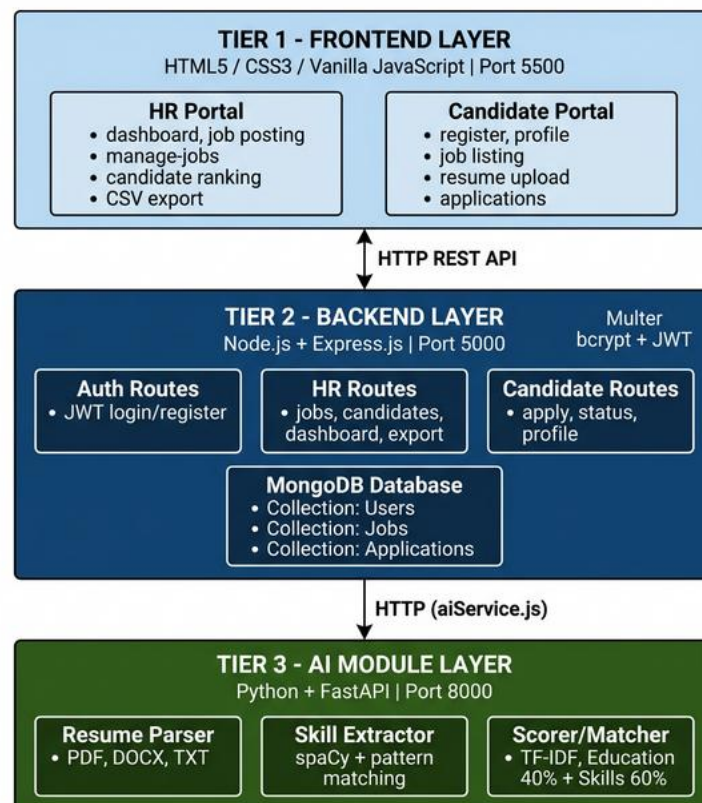


Figure 3.2: System Architecture Diagram

- **Frontend Layer (Port 5500):** The HR and Candidate portals are built with HTML5, CSS3, and Vanilla JavaScript. They communicate with the backend exclusively through REST API calls.
- **Backend Layer (Port 5000):** Node.js/Express.js handles all business logic, authentication (JWT + bcrypt), file uploads (Multer), and database interactions (MongoDB via Mongoose). It also acts as the intermediary between the frontend and the AI module via aiService.js.
- **AI Module Layer (Port 8000):** A Python/FastAPI microservice that receives resume file paths and job requirements, performs NLP-based parsing and analysis, and returns match scores and skill breakdowns.

3.6.2 Data Flow — Candidate Ranking Process

The sequence diagram in Figure 3.3 below illustrates the step-by-step interactions between the system components when HR views the AI-ranked candidate list for a job posting.

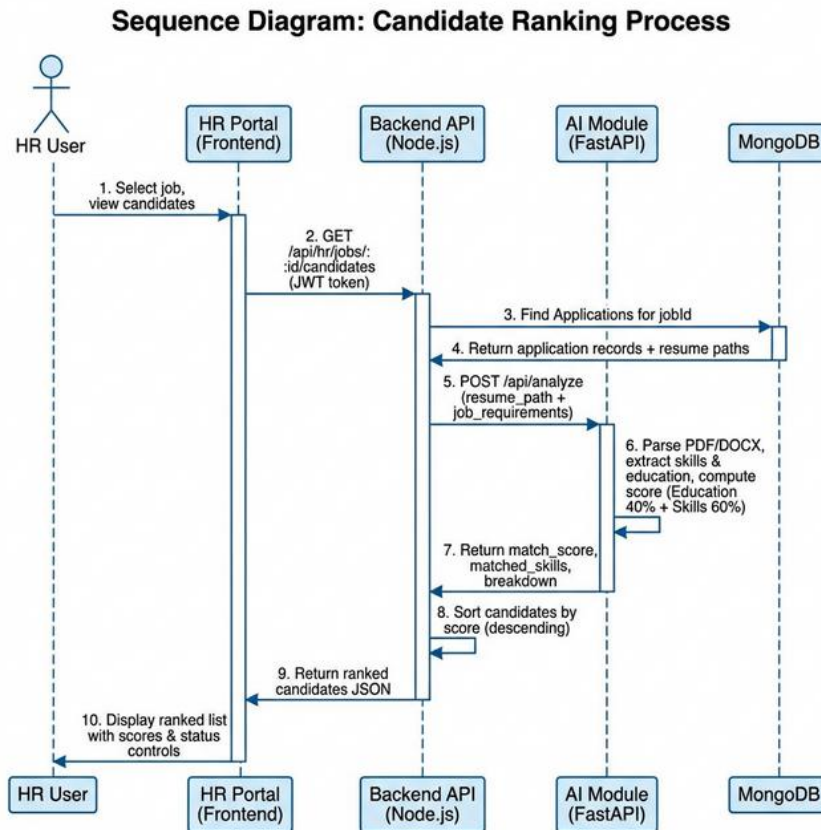


Figure 3.3: UML Sequence Diagram — Candidate Ranking Process

1. HR selects a job on the candidate ranking page
2. The frontend sends GET `/api/hr/jobs/:id/candidates` with the JWT token
3. The backend verifies the token and retrieves all applications from MongoDB
4. For each application, the backend forwards the resume path and job requirements to the AI module via POST `/api/analyze`
5. The AI module parses the resume, extracts skills and education level, and computes a match score
6. The backend sorts of candidates by overall score (descending) and returns the ranked list
7. The frontend renders the ranked candidates with scores, skill breakdowns, and status controls

3.6.3 Activity Diagram

The activity diagram in Figure 3.4 provides a high-level view of the complete system workflow across all three actors — the Candidate, the System/AI, and HR — using parallel swimlanes to show how each party's actions interact throughout the recruitment lifecycle.

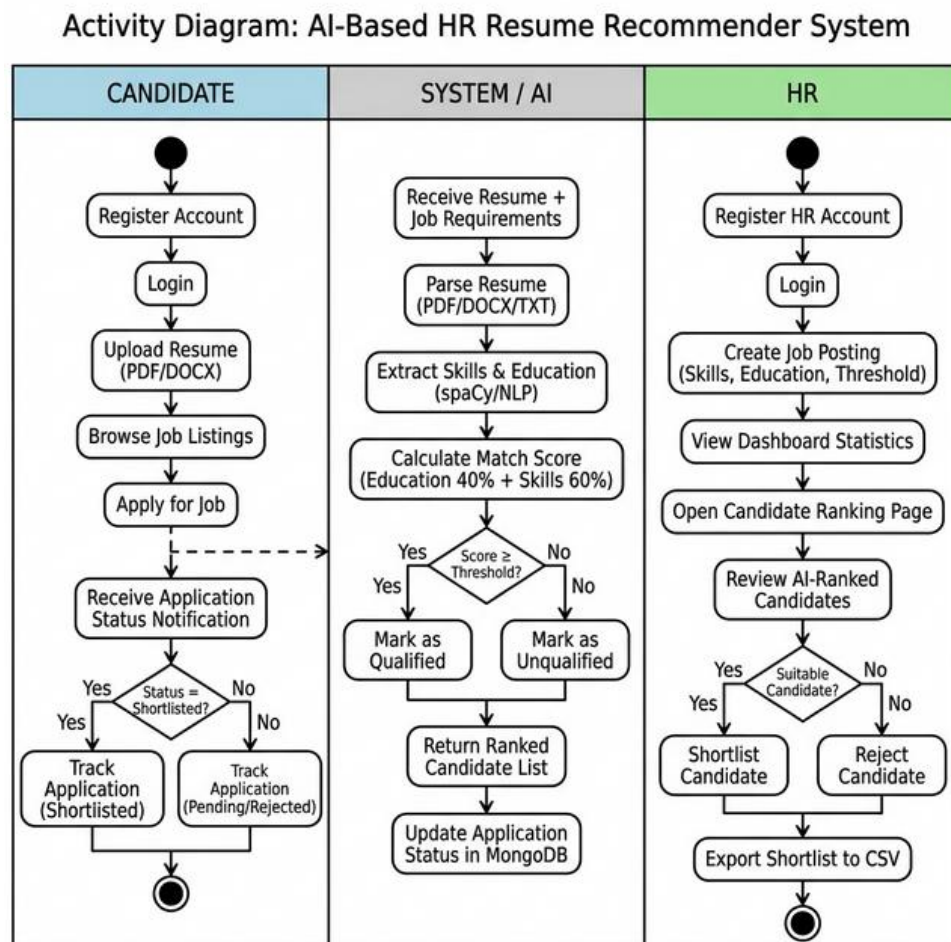


Figure 3.4: UML Activity Diagram — Full System Workflow (Swimlanes)

Key decision points in the workflow:

- **System/AI:** After scoring, candidates whose match score meets the job threshold are marked as Qualified and included in the ranked list shown to HR; those below the threshold are filtered out.
- **HR:** After reviewing the ranked list, HR manually decides to Shortlist or Reject each candidate, and may export the shortlist to CSV for sharing with management.
- **Candidate:** After applying, the candidate tracks the outcome — visible statuses are Pending, Shortlisted, or Rejected.

3.6.4 Database Design (NoSQL — MongoDB)

The system uses three primary MongoDB collections, as illustrated in Figure 3.5 below. As MongoDB is a document-based NoSQL database, there are no enforced foreign key constraints at the database level; relationships between collections are managed at the application layer through Mongoose references (ObjectId references).

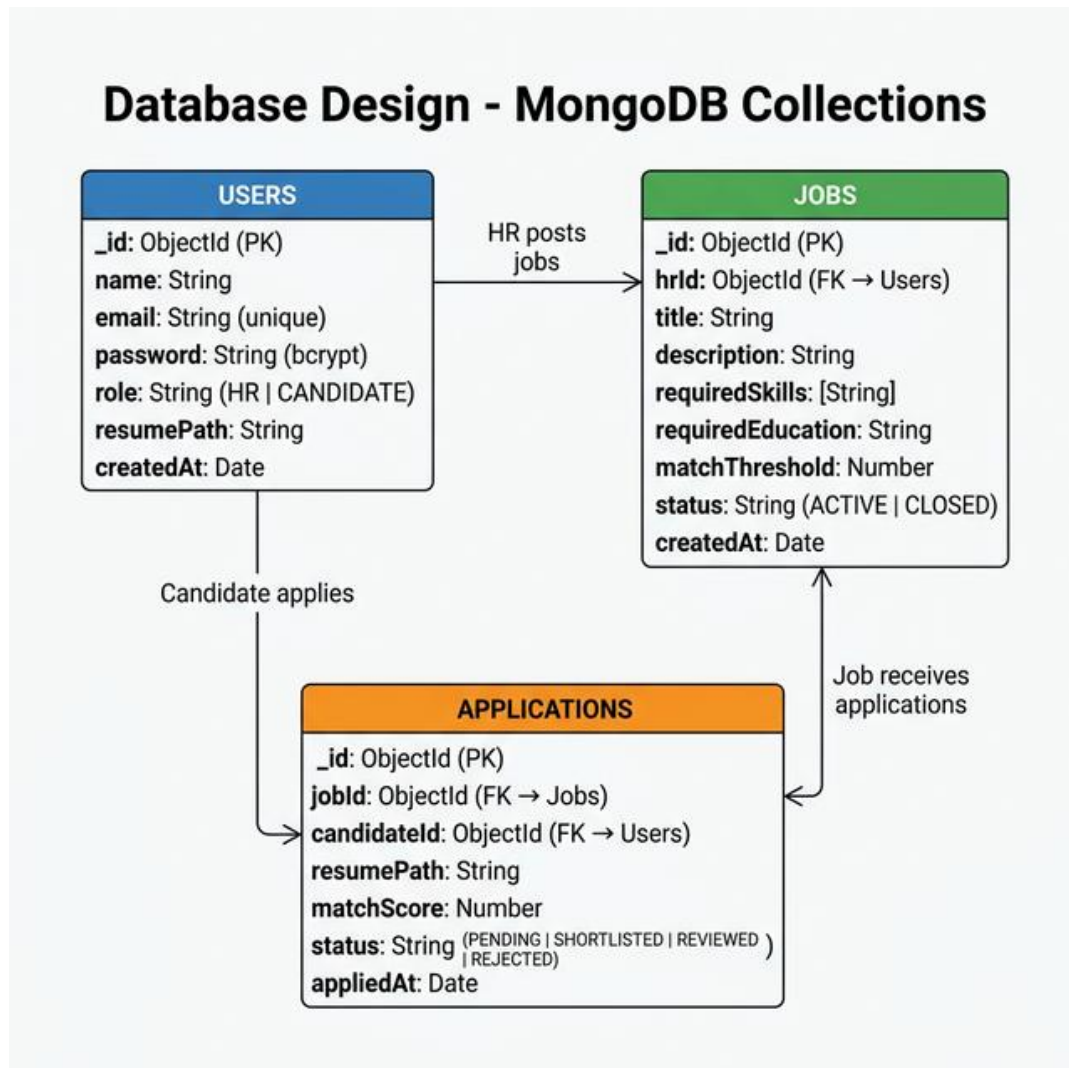


Figure 3.5: MongoDB Collection Schema Diagram

- **Users** — stores both HR and Candidate accounts, distinguished by the role field
- **Jobs** — stores job postings created by HR users, referenced by hrId
- **Applications** — the central join collection linking candidates to jobs, storing the AI match score and application status

3.7 Assumptions and Constraints

The system is assumed to operate in a local development environment (backend: port 5000, frontend: port 5500, AI module: port 8000). Resumes are assumed to be written in English and to be text-based PDFs (not scanned images). HR users are assumed to have basic computer literacy and to enter accurate job requirements, since AI ranking quality depends directly on the quality of inputs provided.

3.7.3 Project Limitations

Table 3.10: Project Limitations

Limitation	Description
No mobile application	The system is a web application only; no iOS or Android app was developed within the project scope
No third-party integration	Integration with JobStreet, Indeed, or LinkedIn was identified as a future enhancement and is not included in the current system
No interview scheduling	Communication and scheduling features between HR and candidates are out of scope
Manual testing only	Automated test suites (e.g., Jest, PyTest) were not implemented due to time constraints; testing was conducted manually
Single-language NLP	The AI module is configured for English-language resumes only; multilingual support is not available
Local deployment	The system is not deployed to a cloud environment; it runs locally and requires all three services to be started manually

Chapter 4: Legal, Social and Professional Issues

This chapter examines the ethical, legal, and professional considerations arising from the development of the AI-Based HR Resume Recommender System — a system that processes personal data and influences hiring decisions.

4.1 Social Issues

4.1.1 User Interaction Implications

For **HR users**, the system shifts workflow from manual resume review to interpreting an AI-generated ranked list. To prevent over-reliance, match scores are presented as a guide only — HR must still review candidates and update statuses manually. All final decisions remain with the human.

For **job candidates**, visible match scores and real-time status tracking (Pending, Shortlisted, Rejected) provide transparency absent from conventional platforms such as JobStreet, empowering candidates to understand how their qualifications align with a role.

4.1.2 Fairness and Algorithmic Bias

The scoring algorithm uses two objective criteria: **education level** (40%) and **skills matching** (60%), applied equally to every candidate regardless of name, gender, ethnicity, or age. A residual bias risk exists in skill extraction — if the pattern library does not recognise non-standard phrasings (e.g., "data wrangling" vs "data analysis"), qualified candidates may score lower than deserved. Additionally, ranking quality depends on the job requirements entered by HR; over-specified requirements will produce biased rankings, reinforcing the importance of responsible system use.

4.1.3 Usability and Accessibility

The system's user interface was designed to be simple and intuitive, requiring no prior technical knowledge to operate. Key design decisions made with usability in mind include:

- **Dual portals with role-based access** — HR and candidates are directed to separate, contextually appropriate interfaces, reducing confusion and the risk of accessing unintended data.
- **Clear visual feedback** — Match scores are displayed as percentage values alongside colour-coded indicators, making the ranking immediately interpretable.
- **Minimal-click workflows** — Core actions such as posting a job, uploading a resume, or updating an application status are all accessible within two clicks.

In terms of **accessibility**, the current system does not fully comply with the Web Content Accessibility Guidelines (WCAG 2.1) — for example, it does not include ARIA labels for all interactive elements or support screen readers. This is noted as a limitation and a future enhancement. The system does, however, use sufficient colour contrast and clear typographic hierarchy to support users with mild visual impairments.

4.1.4 Impact on Different User Groups

Table 4.1: Impact of the System on Different User Groups

User Group	Positive Impact	Potential Risk
HR Professionals	Reduced screening time; consistent, data-driven candidate ranking	Risk of over-reliance on AI scores; reduced critical review of candidate profiles
Job Candidates	Transparent match scores; real-time status tracking	Candidates with non-standard resume formatting may receive lower scores due to parsing limitations
Hiring Managers	CSV export enables efficient sharing of shortlists	Decisions based on AI scores without reviewing full candidate profiles
SMEs & Startups	Accessible, cost-free alternative to enterprise ATS	Requires technical setup (localhost deployment); not plug-and-play for non-technical users

4.2 Legal Issues

4.2.1 Data Protection — Malaysia PDPA 2010

As a system deployed in Malaysia that processes personal data of job candidates, the **Personal Data Protection Act 2010 (PDPA)** is the primary legal framework applicable to this project. The PDPA governs the collection, processing, storage, and use of personal data by organisations in Malaysia.

The following PDPA principles are directly relevant to this system:

Table 4.2: PDPA 2010 Principles and Their Application to This System

PDPA Principle	Application to This System
General Principle	Candidate personal data (name, email, resume, qualifications) is collected and used only for the purpose of job application processing
Notice & Choice Principle	Candidates must be informed at registration that their data will be processed by an AI system for job matching purposes
Disclosure Principle	Candidate data is not disclosed to third parties; only the HR user who posted the relevant job can access a candidate's application and resume
Security Principle	Passwords are hashed using bcrypt; all API routes are protected by JWT authentication; resume files are stored server-side with access controls
Retention Principle	The system retains application and resume data only as long as necessary for the recruitment process; future implementations should include a data deletion mechanism
Data Integrity Principle	User profiles and resumes can be updated by the candidate to ensure data accuracy

4.2.2 International Reference — GDPR

While the General Data Protection Regulation (GDPR) is a European Union regulation and does not directly apply to this system, it represents the international gold standard for data protection and is referenced as best practice. Key GDPR principles such as **data minimisation** (collecting only what is necessary), **purpose limitation** (using data only for its stated purpose), and **right of access** (candidates can view their own data) were used to inform the system's data handling design. Should the system be commercialised and serve users internationally in the future, GDPR compliance would need to be explicitly addressed.

4.2.3 Privacy Considerations

The system handles sensitive personal data including resumes, educational qualifications, and work history. Several privacy considerations were addressed during development:

- **Role-based data isolation:** Candidates can only access their own applications and profile. HR users can only access candidates who have applied to their posted jobs — they cannot browse the full candidate database.
- **Password anonymisation:** No plaintext passwords are stored. All passwords are hashed using bcrypt with appropriate salt rounds before being written to the database.
- **Resume file security:** Resume files are stored in a server-side directory and are not publicly accessible via URL; access is gated through authenticated backend API routes.

A limitation of the current implementation is the **absence of explicit user consent prompts** (e.g., a checkbox accepting terms of data processing) at the registration stage. This is a compliance gap relative to both PDPA and GDPR requirements and is identified as a priority enhancement for future versions.

4.2.4 Intellectual Property

All code produced for this project is original work authored by the developer. The following third-party libraries and frameworks are used under their respective open-source licences:

Table 4.3: Third-Party Libraries and Their Open-Source Licences

Technology	Licence
Node.js / Express.js	MIT Licence
MongoDB / Mongoose	Apache 2.0 / MIT Licence
Python / FastAPI	MIT Licence
spaCy	MIT Licence
Chart.js	MIT Licence
PyPDF2	BSD Licence

All licences listed above permit free use, modification, and distribution for academic and commercial purposes, subject to attribution requirements. No proprietary or commercially restricted libraries were used in this project.

The system does not use any third-party resume datasets or pre-trained models beyond spaCy's standard English NLP model (en_core_web_sm), which is distributed under the MIT Licence. No data was scraped from external platforms, and all test resumes used during development were either publicly available sample documents or created specifically for testing purposes.

4.3 Professional Issues

4.3.1 Ethical Guidelines — ACM and BCS

As a computing professional, this project was developed with reference to established ethical guidelines from professional bodies:

ACM Code of Ethics (2018)

The Association for Computing Machinery (ACM) Code of Ethics provides principles for responsible computing practice. The following principles are particularly relevant:

- *"Contribute to society and to human well-being"* — The system is designed to benefit both HR professionals (efficiency) and candidates (transparency), contributing positively to the recruitment ecosystem.
- *"Avoid harm"* — The system's design avoids automating final rejection decisions, ensuring human oversight is retained in all hiring outcomes.
- *"Be honest and trustworthy"* — Match scores are computed from an objective, documented algorithm, ensuring transparency in how candidates are evaluated.
- *"Respect privacy"* — Role-based access control and data isolation ensure that candidate data is handled appropriately.

BCS Code of Conduct

The British Computer Society (BCS) Code of Conduct similarly emphasises public interest, competence, integrity, and professionalism. This project upholds these principles through:

- Ensuring the system is fit for purpose through structured testing (Section 3.5)
- Documenting all known limitations honestly (Section 3.7)
- Treating user data with appropriate care and security measures

4.3.2 Security Considerations

Security was a non-functional requirement of this system from the outset. The following security measures were implemented:

- **Authentication:** JWT (JSON Web Tokens) are issued on login with a configurable expiry period. All protected routes verify the token on every request.
- **Authorisation:** Role-based access control (`requireRole('HR') / requireRole('CANDIDATE')`) ensures that HR and candidate routes are strictly segregated.
- **Password security:** All passwords are hashed using bcrypt before storage; raw passwords are never logged or persisted.
- **Input validation:** The backend validates incoming data types and required fields using Mongoose schema enforcement, reducing the risk of malformed data being stored.
- **File upload security:** Multer is configured to accept only specific MIME types (PDF, DOCX, TXT) and enforces a maximum file size, preventing malicious file uploads.

Remaining security limitations include the absence of HTTPS in the local development environment, no rate limiting on API endpoints, and no CSRF protection. These would need to be addressed before any production deployment.

4.3.3 Responsible Software Development

This project was developed with a commitment to responsible software engineering practices:

- **Transparency in AI decisions:** The scoring algorithm is fully documented and deterministic — the same resume and job requirements will always produce the same score. This avoids the "black box" problem common in machine learning systems.
- **Human oversight retained:** The system assists HR in decision-making but does not replace human judgement. All shortlisting and rejection decisions are made manually by HR users.
- **Honest representation of limitations:** Known limitations — including the English-only NLP model, the lack of WCAG compliance, and the manual testing approach — are documented openly in this report rather than omitted.
- **Use of Generative AI tools:** During the development of this project, Generative AI tools (specifically Google's Antigravity AI assistant) were used to support the drafting and refinement of this report. All AI-generated content was reviewed, critically evaluated, and edited to ensure accuracy and alignment with the actual system. The use of these tools is disclosed in compliance with the MAL3018 module requirement to document all tools used.

4.3.4 API Usage and Licensing

All external APIs and libraries used in this project are open-source and appropriately licensed (as detailed in Section 4.2.4). No proprietary external APIs requiring commercial licensing were integrated. The system does not currently connect to any external services such as jobstreet, Indeed, or LinkedIn APIs — these integrations are noted as future enhancements but would require review of those platforms' terms of service and API usage policies before implementation.

Chapter 5: Project Management

This chapter documents how the project was planned, managed, and monitored, covering tools used, the Agile methodology adopted, sprint structure, and monitoring and control mechanisms.

5.1 Project Management Tools

5.1.1 Tools Used

The following tools were used to plan, track, and manage the development of this project:

Table 5.1: Project Management and Development Tools

Tool	Category	Purpose
GitHub	Version Control	Hosted the project repository at ImanHaqimi/hr-resume-recommender; used for source code management, branching, and commit history
GitHub Projects	Task Tracking	Used as the primary task board to organise and track development tasks across sprints using a Kanban-style board with columns: Backlog, In Progress, and Done
Visual Studio Code	Development IDE	Primary code editor for frontend (HTML/CSS/JS), backend (Node.js), and AI module (Python) development
Postman	API Testing	Used to test and validate backend REST API endpoints during development, prior to frontend integration
FastAPI Swagger UI	API Documentation	Auto-generated interactive documentation at http://localhost:8000/docs , used to test the AI module's endpoints
MongoDB Compass	Database Management	GUI tool used to inspect and validate MongoDB collections during development and debugging

5.1.2 Task Tracking and Prioritisation

GitHub Projects was selected over dedicated project management tools such as Trello or Jira due to its direct integration with the GitHub repository, enabling tasks (issues) to be linked directly to code commits and pull requests. This tight integration ensured that every task tracked on the board corresponded to a specific, traceable change in the codebase.

Tasks were organised into the following priority tiers based on the project's MVP definition (Section 3.2.4):

- **High Priority (MVP):** Core features required for a functional, minimally viable system — HR and Candidate authentication, job posting, resume upload, AI scoring, and candidate ranking.
- **Medium Priority (Enhancement):** Features planned for later sprints — CSV export, manage-jobs page, candidate profile management, dashboard visualisations.
- **Low Priority (Future):** Features deferred beyond the project scope — WCAG compliance, mobile responsiveness, third-party job board integration, OCR for image-based PDFs.

This tiered prioritisation allowed the project to maintain focus on delivering a working system first, with enhancements added incrementally once the MVP was stable.

5.2 Methodology

5.2.1 Choice of Methodology — Agile (Scrum)

The project adopted **Agile (Scrum)** over Waterfall because several features (dual dashboard charts, CSV export, candidate profile) were not fully defined at project initiation and emerged iteratively. Waterfall's sequential structure would have delayed AI-backend integration testing until project end, increasing risk. Scrum's sprint cycle — with defined goals, a prioritised backlog, and an end-of-sprint review — provided a disciplined, auditable framework appropriate for a time-boxed academic project.

5.2.2 Sprint Planning

The project was divided into **five sprints**, each lasting approximately **two to three weeks**, aligned with the 20-week project timeline. Each sprint had a defined goal and a set of backlog items drawn from the prioritised task board.

Table 5.2: Sprint Plan and Deliverables

Sprint	Duration	Goal	Key Deliverables
Sprint 1	Weeks 1–3	Project Setup & Authentication	Project repository setup, Node.js/Express backend skeleton, MongoDB connection, User registration & login with JWT, Basic frontend portal pages
Sprint 2	Weeks 4–7	Core Recruitment Features	Job posting (HR), Job listing (Candidate), Resume upload (PDF/DOCX), AI module skeleton (FastAPI + spaCy setup), Backend-AI integration (aiService.js)
Sprint 3	Weeks 8–11	AI Scoring & Candidate Ranking	Resume parser (PyPDF2, python-docx), Skill extractor, TF-IDF scoring engine, Candidate ranking page (HR), Application status update (Shortlist/Reject), Candidate application tracking
Sprint 4	Weeks 12–15	Enhancements & Dashboard	HR dashboard with summary statistics and Chart.js visualisations (donut chart + status bars), manage-jobs page, Candidate profile management, CSV export
Sprint 5	Weeks 16–20	Testing, Polish & Documentation	Full UAT, bug fixes, UI polish, README, project report writing (Chapters 1–10)

5.2.3 Project Gantt Chart

Figure 5.1 below shows the project timeline as planned at the beginning of the project and documented in the project proposal. The chart covers the full 20-week development period and maps each major project phase to its allocated weeks.

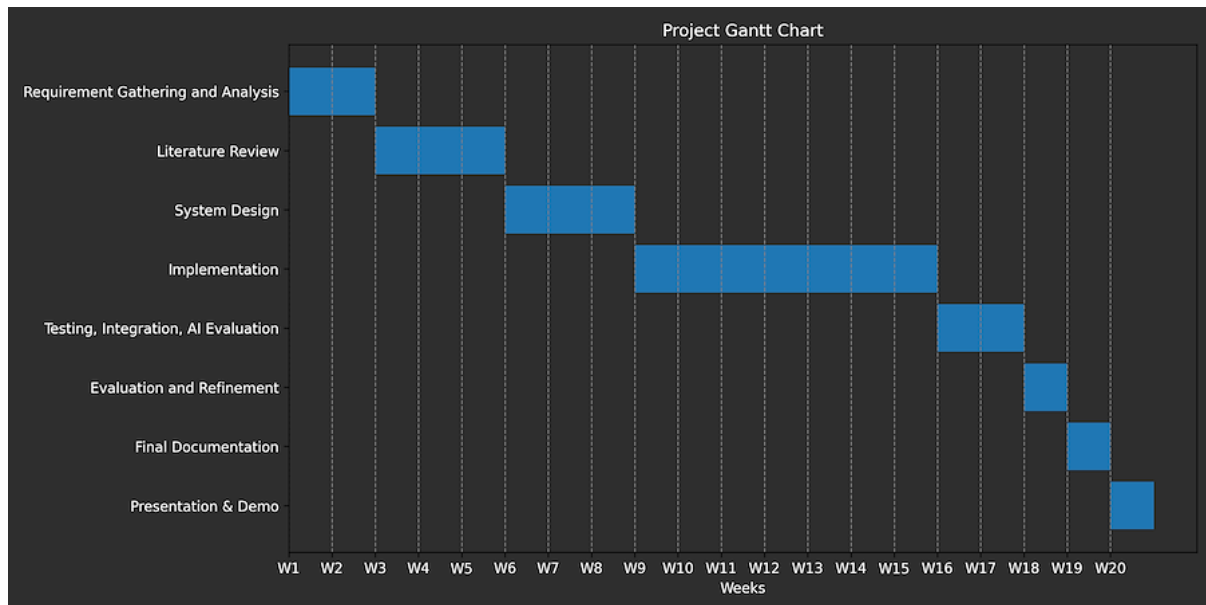


Figure 5.1: Project Gantt Chart — Planned Timeline (20 Weeks)

The actual development closely followed this planned timeline. The main deviation was in **Weeks 9–11 (Implementation phase)**, where additional time was required to resolve AI-backend integration issues (as detailed in Section 5.3.2). This was absorbed by reducing the time allocated to System Design documentation in that period, without impacting the final submission deadline. The Testing and Final Documentation phases proceeded on schedule.

5.2.4 Backlog Management

The product backlog was maintained as a list of GitHub issues on the GitHub Projects board. Each issue was written in the form of a user story (e.g., *"As an HR user, I want to view ranked candidates so I can identify the best applicants quickly"*) and labelled with its priority tier (MVP, Enhancement, Future) and sprint assignment.

At the start of each sprint, the sprint backlog was drawn from the top of the prioritised product backlog. New items identified during the sprint (e.g., bug reports, scope changes) were added to the product backlog and prioritised for a subsequent sprint rather than being added mid-sprint, preserving sprint focus.

5.3 Monitoring and Control

5.3.1 Sprint Reviews

At the end of each sprint, a review assessed whether the sprint goal was met: each feature was tested end-to-end, progress was discussed with the project supervisor, and incomplete items were returned to the backlog and re-prioritised for the next sprint.

5.3.2 Handling Delays and Scope Changes

As a solo developer, delays and scope changes were inevitable. The following strategies were employed to manage these effectively:

Delays encountered:

- **AI-backend integration (Sprint 2–3):** The initial integration of the FastAPI AI module with the Node.js backend took longer than planned due to CORS configuration issues and differences in how file paths were handled between the two services. This was resolved by Sprint 3 through the implementation of `aiService.js` as a dedicated service layer. The delay was absorbed without impacting the overall timeline by reducing the scope of frontend polish in that sprint.
- **Resume parsing accuracy (Sprint 3):** Initial PDF text extraction produced inconsistent results for PDFs generated by different tools. A fallback chain (`PyPDF2` → `pdfminer` → raw text extraction) was implemented to improve reliability.

Scope changes:

- The manage-jobs page and CSV export features were not in the original MVP but were added in Sprint 4 following the stakeholder interview, which confirmed their operational importance.
- The "Reviewed" application status was removed from the candidate-facing filter after user testing revealed it caused confusion — candidates could not distinguish between "reviewed but not yet decided" and "shortlisted". The status remains internally but is hidden from the candidate view.

All scope changes were evaluated against the project's remaining timeline before being accepted into the backlog, ensuring deliverability within the submission deadline.

5.3.3 Time Management and Workload Distribution

As a solo project, time management was critical. Key practices: fixed weekly development hours; documentation written in parallel with development rather than deferred to the final sprint; MVP-first discipline ensuring core features were always working before enhancements; and high-risk tasks (AI integration) scheduled early to allow resolution time before submission.

Table 5.3: Estimated Time Allocation by Activity

Activity	Estimated Time
Backend development (Node.js, Express.js, MongoDB)	30%
AI module development (Python, FastAPI, spaCy, TF-IDF)	25%
Frontend development (HTML, CSS, JavaScript)	20%
Testing and debugging	10%
Project report writing	10%
Project planning and management	5%

Chapter 6: Project Stages / Development Phases

This chapter documents the complete development lifecycle of the AI-Based HR Resume Recommender System, divided into five stages from planning through to final delivery, mapped to the Agile sprint plan in Chapter 5.



Figure 6.1: Project Development Lifecycle — 5-Stage Process

Table 6.1: Summary of Development Stages

Stage	Phase	Weeks	Key Activities
Stage 1	Planning & Preparation	W1–W5	Problem analysis, literature review, stakeholder interview, proposal
Stage 2	Design & Setup	W5–W9	System architecture, UI mockups, environment and toolchain setup
Stage 3	Implementation	W9–W15	Core feature development, AI module, backend and database integration
Stage 4	Testing & Refinement	W15–W18	Bug fixing, performance improvements, user acceptance testing
Stage 5	Finalisation	W18–W20	Final testing, documentation, report writing, demo preparation

6.1 Stage 1 — Planning and Preparation (Weeks 1–5)

6.1.1 Problem Analysis

The project began with analysis of the problem domain through a review of existing literature (Salton & Buckley, 1988; Bogen & Rieke, 2018; LinkedIn Talent Solutions, 2025) and a structured stakeholder interview with Mrs Tracy, HR Executive at PKT (Section 2.1.5). Five core problems were identified (Section 1.7): time-consuming manual screening, evaluation inconsistency, lack of candidate transparency, difficulty managing multiple job postings, and the absence of a structured shortlisting workflow.

6.1.2 Literature Review

A review of academic literature was conducted covering AI-based resume screening, NLP techniques, and ethical considerations (detailed in Section 2.1.4). This informed the technology selection (spaCy, TF-IDF, FastAPI) and the decision to adopt a rule-based, interpretable scoring model.

6.1.3 Initial Design and Proposal

The planning stage culminated in the submission of a formal project proposal, which included:

- A defined project scope, aims, and SMART objectives
- An initial system architecture concept (three-tier: Frontend → Backend → AI Module)
- A risk register with seven identified risks and mitigation strategies
- A project Gantt chart covering the 20-week development timeline (Figure 5.1)
- A Minimum Viable Product (MVP) definition to guide development prioritisation

6.2 Stage 2 — Design and Setup (Weeks 5–9)

6.2.1 System Architecture

Following the proposal, the three-tier microservice architecture was designed in detail (documented in Section 3.6.1, Figure 3.2). Key decisions: the AI module as a standalone FastAPI microservice on Port 8000; MongoDB for schema flexibility; and JWT-based authentication for stateless API security across both portals.

6.2.2 UI Mockups and Interface Design

Before implementation began, the key user interface screens were designed conceptually. The following pages were planned for the HR portal and Candidate portal:

HR Portal (planned):

- Login / Register page
- Dashboard with summary statistics
- Candidate ranking page (per job)
- Manage-jobs page
- Application status update controls

Candidate Portal (planned):

- Login / Register page
- Profile management page
- Job listing and search page
- Application history with status and match scores

The UI design prioritised simplicity and function — HR users require efficiency and data density (sortable tables, charts), while candidates require clarity and transparency (visible scores, clear status indicators). A consistent navigation structure (sidebar) was planned for both portals to reduce cognitive load.

Figure 6.2 below shows the implemented HR Dashboard — the primary landing page for HR users after login — which closely reflects the original design concept, with summary cards, a donut chart, and horizontal status bars.

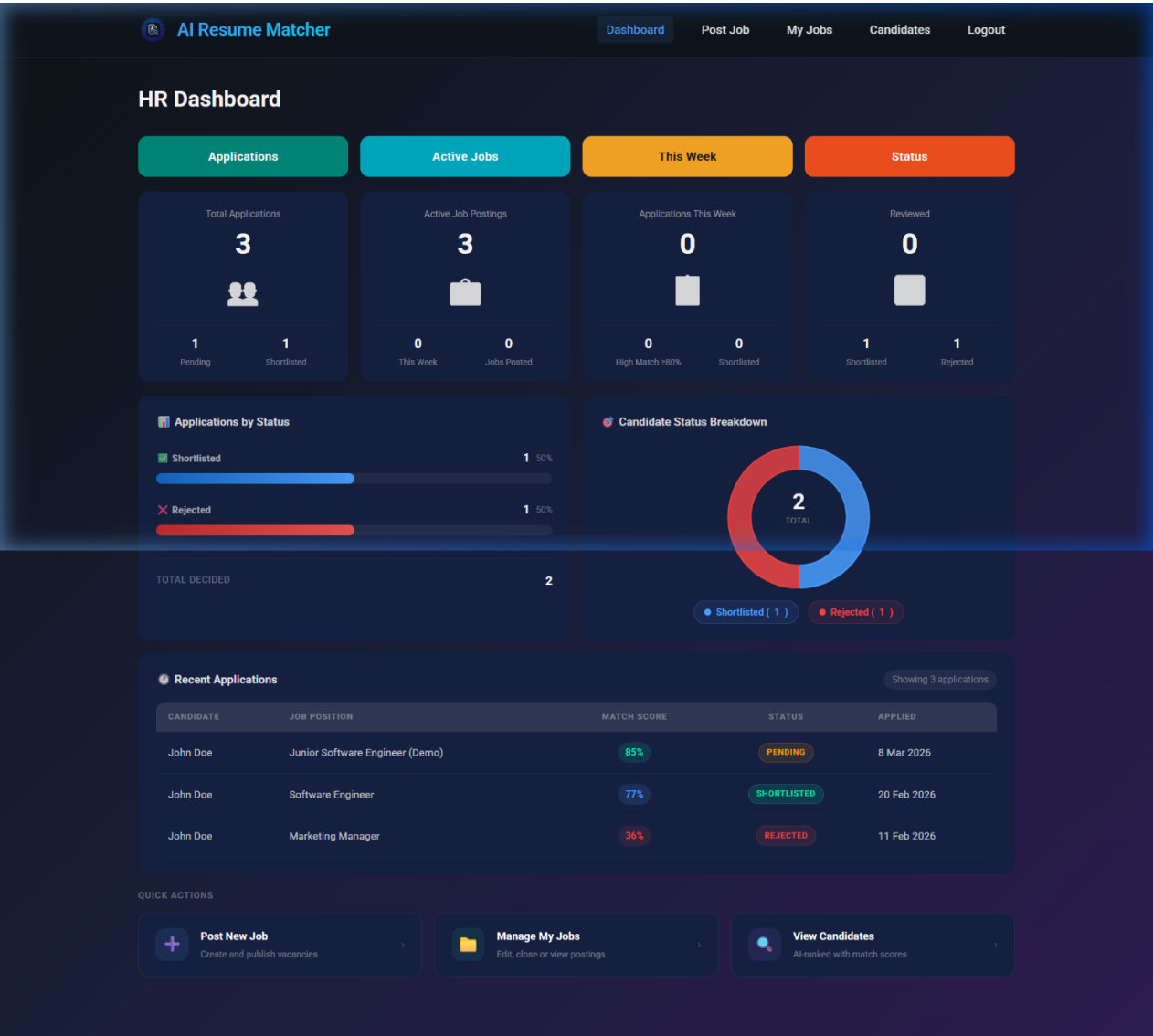


Figure 6.2: HR Dashboard — Summary Cards and Application Status Visualisations

6.2.3 Environment Setup

The development environment was configured at this stage:

- **Node.js v18** and **npm** installed for the backend
- **Python 3.10** with a virtual environment (venv) configured for the AI module
- **MongoDB** (local instance) connected via MongoDB Compass for inspection during development
- **Visual Studio Code** configured with relevant extensions (ESLint, Pylint, REST Client)
- **GitHub repository** initialised (ImanHaqimi/hr-resume-recommender) with .gitignore and initial project structure (/frontend, /backend, /ai-module)
- **Environment variables** defined in .env files for MongoDB URI, JWT secret, and AI module URL

6.3 Stage 3 — Implementation (Weeks 9–15)

Stage 3 was the longest and most intensive phase, covering the full development of all three system tiers: backend, AI module, and frontend.

6.3.1 Backend Development (Node.js / Express.js)

The backend was developed first, as it underpins all frontend interactions and AI module communication. Development followed a route-based modular structure:

- **Authentication routes** (/api/auth/) — User registration and login with bcrypt password hashing and JWT token issuance
- **Job routes** (/api/jobs/) — CRUD operations for job postings, accessible only to HR users
- **Application routes** (/api/applications/) — Resume upload (Multer), application creation, and status updates
- **HR routes** (/api/hr/) — Candidate ranking per job, status management (Shortlist/Reject), and CSV export
- **AI service** (aiService.js) — A dedicated module that forwards resume file paths and job requirements to the FastAPI AI module via HTTP and processes the returned scores

Middleware layers implemented include JWT verification (protect), role-based access control (requireRole), and Multer file upload validation (MIME type and size limits).

6.3.2 AI Module Development (Python / FastAPI / spaCy)

The AI module was developed in parallel with the backend, exposing a single primary endpoint (POST /api/analyze) that accepts a resume file path and job requirements and returns a structured match result.

The AI processing pipeline consists of three stages:

1. **Resume Parser** — Extracts raw text from PDF (PyPDF2 / pdfminer), DOCX (python-docx), or TXT files
2. **Skill & Education Extractor** — Uses spaCy's pattern matching (PhraseMatcher) to identify skills from a pre-defined skill library, and rule-based keyword detection to classify education level (Diploma, Bachelor's, Master's, PhD)
3. **Scoring Engine** — Computes a match score using a weighted formula: $(\text{Education Score} \times 0.4) + (\text{Skill Score} \times 0.6)$, where skill score = matched skills / required skills

The AI module was tested independently using the FastAPI Swagger UI (<http://localhost:8000/docs>) before integration with the backend.

6.3.3 Frontend Development (HTML / CSS / JavaScript)

The frontend was developed after the backend APIs were stable enough to integrate against. Both the HR and Candidate portals were built using HTML5, CSS3, and Vanilla JavaScript. Key pages implemented:

HR Portal:

- login.html / register.html — Authentication forms with JWT storage in localStorage
- dashboard.html — Summary cards (total jobs, applications, shortlisted), donut chart (Shortlisted vs Rejected via Chart.js), horizontal status bars
- manage-jobs.html — Job listing with edit/delete, visual skill tags, match threshold setting
- candidates.html — AI-ranked candidate table with match score, skill breakdown, status dropdown
- view-resume.html — Inline resume viewer

Candidate Portal:

- login.html / register.html — Authentication forms
- profile.html — Profile editing form with resume upload
- jobs.html — Available job listings with apply functionality
- applications.html — Application history with AI match score and status filter (Pending / Shortlisted / Rejected)

Figure 6.3 shows the Candidate Ranking page — the core HR workflow screen — displaying AI-ranked candidates with match scores, education levels, matched skills, and status controls.

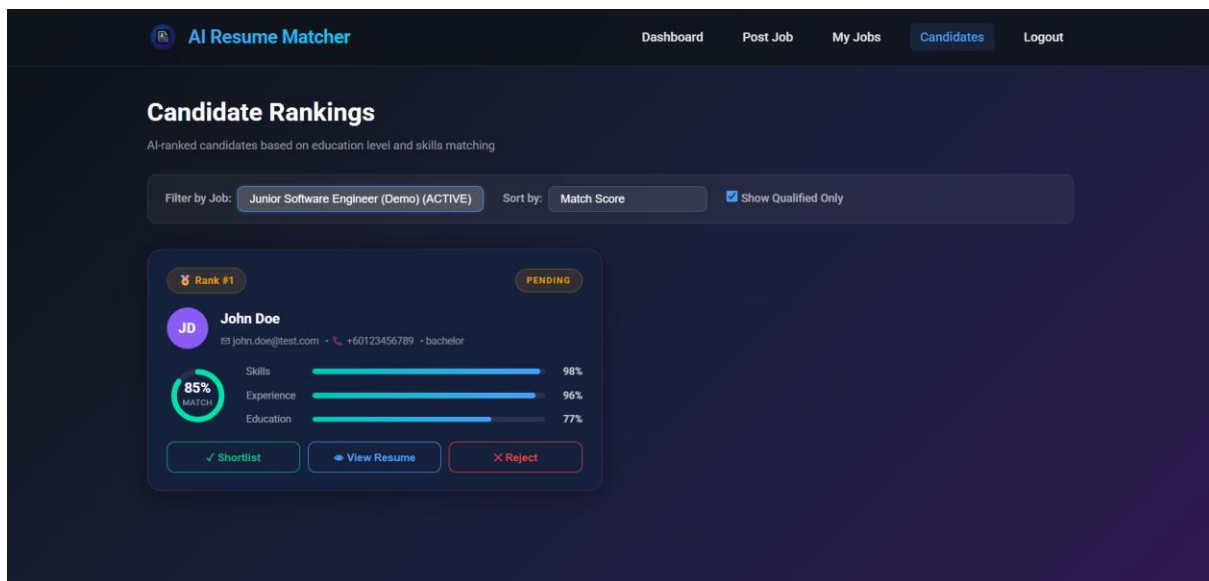


Figure 6.3: HR Candidate Ranking Page — AI-Ranked List with Match Scores and Status Controls

Figure 6.4 shows the Candidate Applications page, where candidates can view all their submitted applications, filtered by status, with the AI match score displayed for each job.

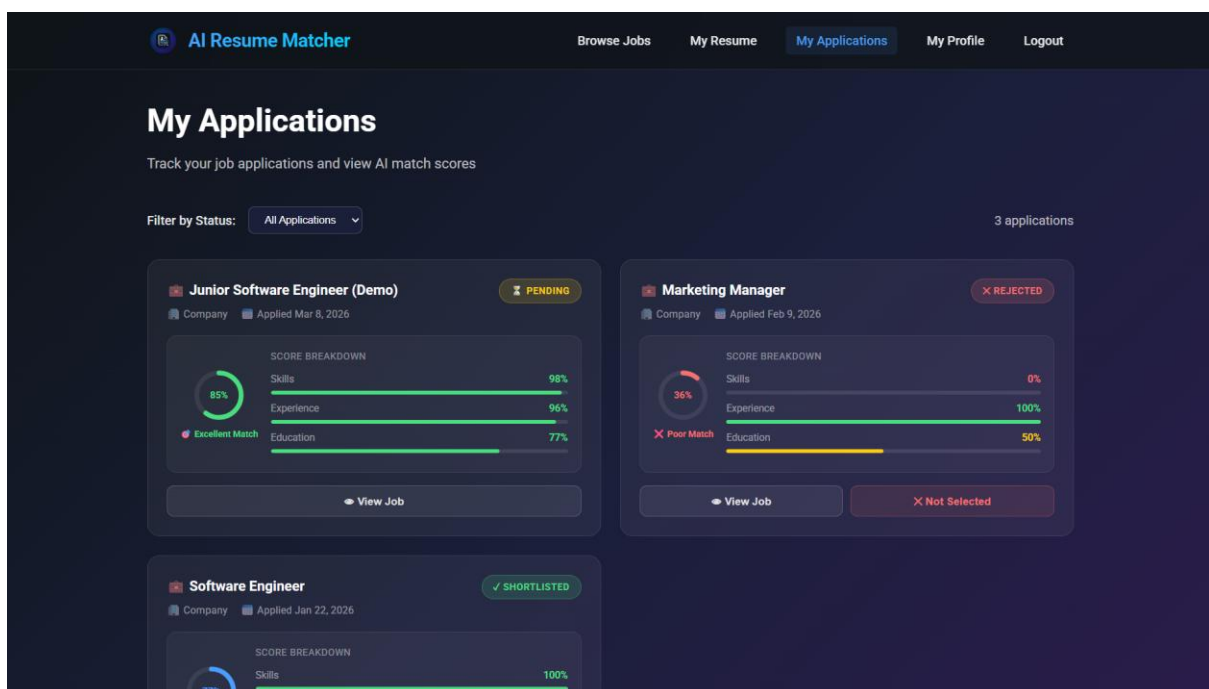


Figure 6.4: Candidate Portal — My Applications Page with Match Scores and Status Filter

6.4 Stage 4 — Testing and Refinement (Weeks 15–18)

6.4.1 Bug Fixing

Following the completion of initial implementation, a structured testing phase was conducted.

Key bugs identified and resolved included:

Table 6.2: Key Bugs Identified and Fixed During Testing

#	Bug Observed	Root Cause	Fix Applied
1	AI module returning 500 error for some PDF resumes	PyPDF2 failing silently on certain PDF encodings	Added pdfminer as a fallback parser; implemented try/except chain
2	Candidate status not updating on candidate's page after HR update	Frontend loading stale cached data	Added cache-busting query string to API calls
3	Manage-jobs page showing jobs from all HR users	Missing user-scoped filter on jobs query	Added hrId: req.user.id filter to jobs GET route
4	CSV export including internal score fields not needed by end user	Over-broad data serialisation	Updated CSV export to include only candidate name, email, score, status, and matched skills
5	Dashboard donut chart not rendering on first load	Chart.js initialising before API data returned	Moved chart initialisation inside the API response callback

6.4.2 Performance Improvements

Several performance enhancements were applied following initial testing:

- **Lazy loading of candidate scores:** The AI scoring call was moved to be triggered only when HR views the candidate ranking page for a specific job, rather than being run for all jobs on dashboard load
- **Debounced search:** The job search input on the candidate portal was debounced (300ms delay) to prevent excessive API calls on every keystroke
- **Pagination placeholder:** While full pagination was not implemented within the project scope, the backend now returns a maximum of 50 candidates per job to prevent excessive data transfer

6.4.3 User Testing Feedback

User Acceptance Testing (UAT) was conducted with two test accounts (one HR and one Candidate) following the test plan in Section 3.5.3 (Table 3.9). All 7 UAT test cases passed. Key feedback observations from the testing process:

- The "Reviewed" status was removed from the candidate-facing status filter as it caused confusion — candidates could not meaningfully distinguish between "Reviewed but not decided" and "Shortlisted". The status was retained internally but hidden from candidate view.
- The match score colour coding (green/amber/red based on score range) was added to the candidate ranking table to improve at-a-glance readability for HR users.

6.5 Stage 5 — Finalisation (Weeks 18–20)

6.5.1 Final Testing

A final round of end-to-end testing was conducted covering the complete user journeys for both HR and Candidate users. A dedicated demo scenario was set up using a scripted seed dataset (Junior Software Engineer job + three candidate profiles with varied qualifications) to reliably demonstrate the AI ranking functionality during the project presentation.

6.5.2 Documentation

All project documentation was completed in this stage:

- **README.md** — Comprehensive setup guide including environment configuration, dependency installation, and step-by-step instructions for running all three services
- **API documentation** — Auto-generated via FastAPI Swagger UI and referenced in the README
- **GitHub repository** — All code committed with descriptive messages; repository made public for submission

6.5.3 Report Writing

The project report was written progressively across all stages (following the documentation-in-parallel principle from Section 5.3.3), with final chapters (Chapters 7–10) completed in Stage 5. The report is structured across 10 chapters as outlined in Section 1.8.

6.6 Stage Overview — Reflection on Progress

6.6.1 Progress Across Stages

Overall, the five-stage development lifecycle was followed as planned, with all core MVP features delivered by the end of Stage 3. The decision to prioritise backend and AI module development before the frontend proved effective — having stable, tested APIs to build against significantly reduced frontend debugging time.

The most challenging stage was **Stage 3 (Implementation)**, primarily due to the complexity of integrating the Python AI module with the Node.js backend. Cross-language communication, file path handling differences between the two environments, and CORS configuration issues all required additional effort beyond initial estimates.

Stage 4 (Testing) was more productive than anticipated — the structured UAT test plan (Table 3.9) provided clear pass/fail criteria, and all bugs were resolved within the allocated testing window.

6.6.2 Lessons Learned

- **Define API contracts early:** Establishing the request/response format for the AI module endpoint before implementation began would have prevented several integration debugging sessions.
- **Test cross-service communication earlier:** Integration between the backend and AI module should have been smoke-tested in Sprint 1/2, not left until the full implementation was complete.
- **Parallelise documentation with development:** Maintaining running notes on design decisions throughout development made final report writing significantly faster and more accurate.

Chapter 7: Output Documentation

This chapter presents the final technical outputs of the AI-Based HR Resume Recommender System at point of submission, documenting completed artefacts and comparing implementation against the original design.

7.1 Output Diagrams

7.1.1 Final System Architecture Diagram

The system architecture that was designed at the outset and implemented throughout development is shown in Figure 7.1. The final architecture is a **three-tier microservice design**, consistent with the original plan. Each tier runs as an independent service on a dedicated port, communicating via HTTP REST APIs.

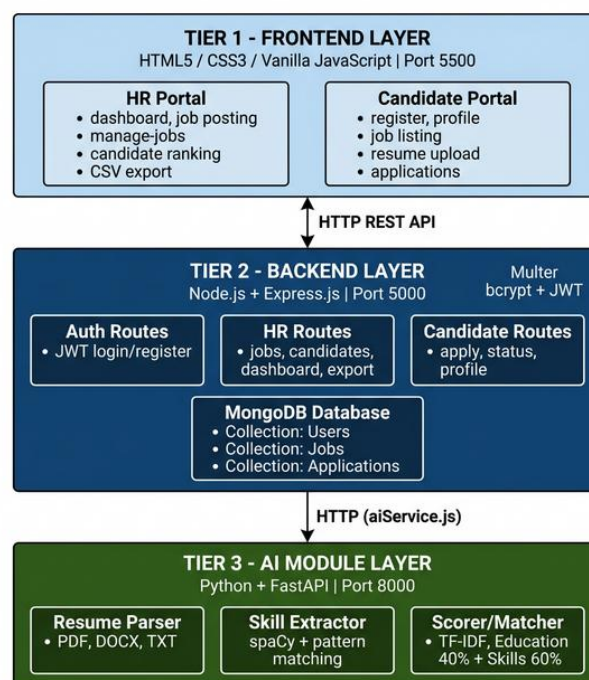


Figure 7.1: Final System Architecture Diagram — Three-Tier Microservice Design

Tier	Technology	Port	Role
Frontend	HTML5, CSS3, Vanilla JavaScript	5500	HR Portal and Candidate Portal; communicates with backend via REST API calls

Backend	Node.js, Express.js, MongoDB (Mongoose)	5000	Business logic, authentication, file handling, AI service proxy
AI Module	Python, FastAPI, spaCy, TF-IDF	8000	Resume parsing, skill extraction, scoring, and candidate ranking

7.1.2 Final UML Diagrams

Sequence Diagram — Candidate Ranking Process

Figure 7.2 shows the UML Sequence Diagram for the candidate ranking process — the core workflow of the system. This diagram reflects the actual implemented behaviour, including the aiService.js proxy layer that mediates between the backend and AI module.

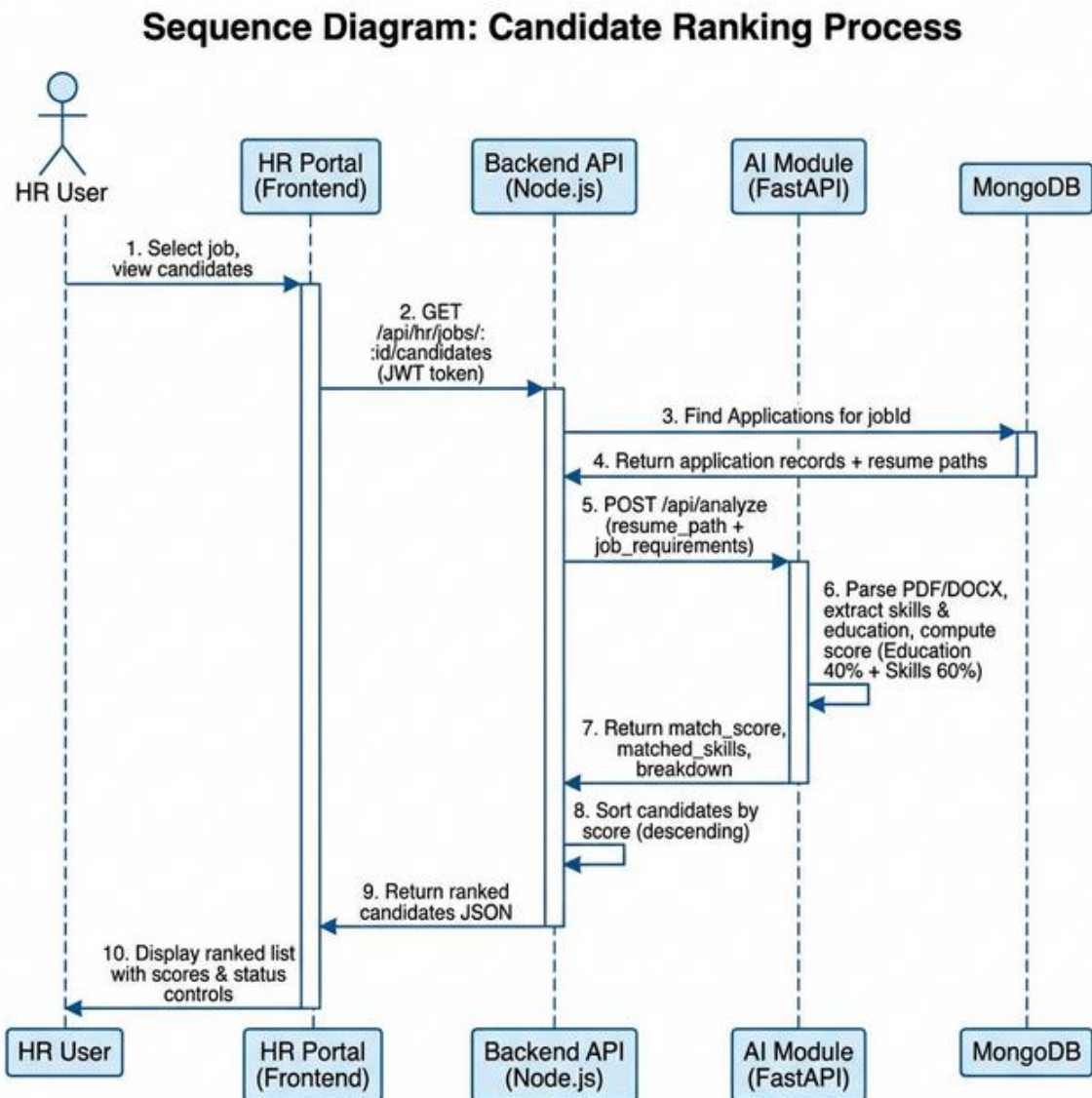


Figure 7.2: UML Sequence Diagram — Candidate Ranking Process (Final Implementation)

Activity Diagram — Full System Workflow

Figure 7.3 shows the UML Activity Diagram for the complete system workflow across all three actors — Candidate, System/AI, and HR — using swimlanes to illustrate parallel activities and handoff points.

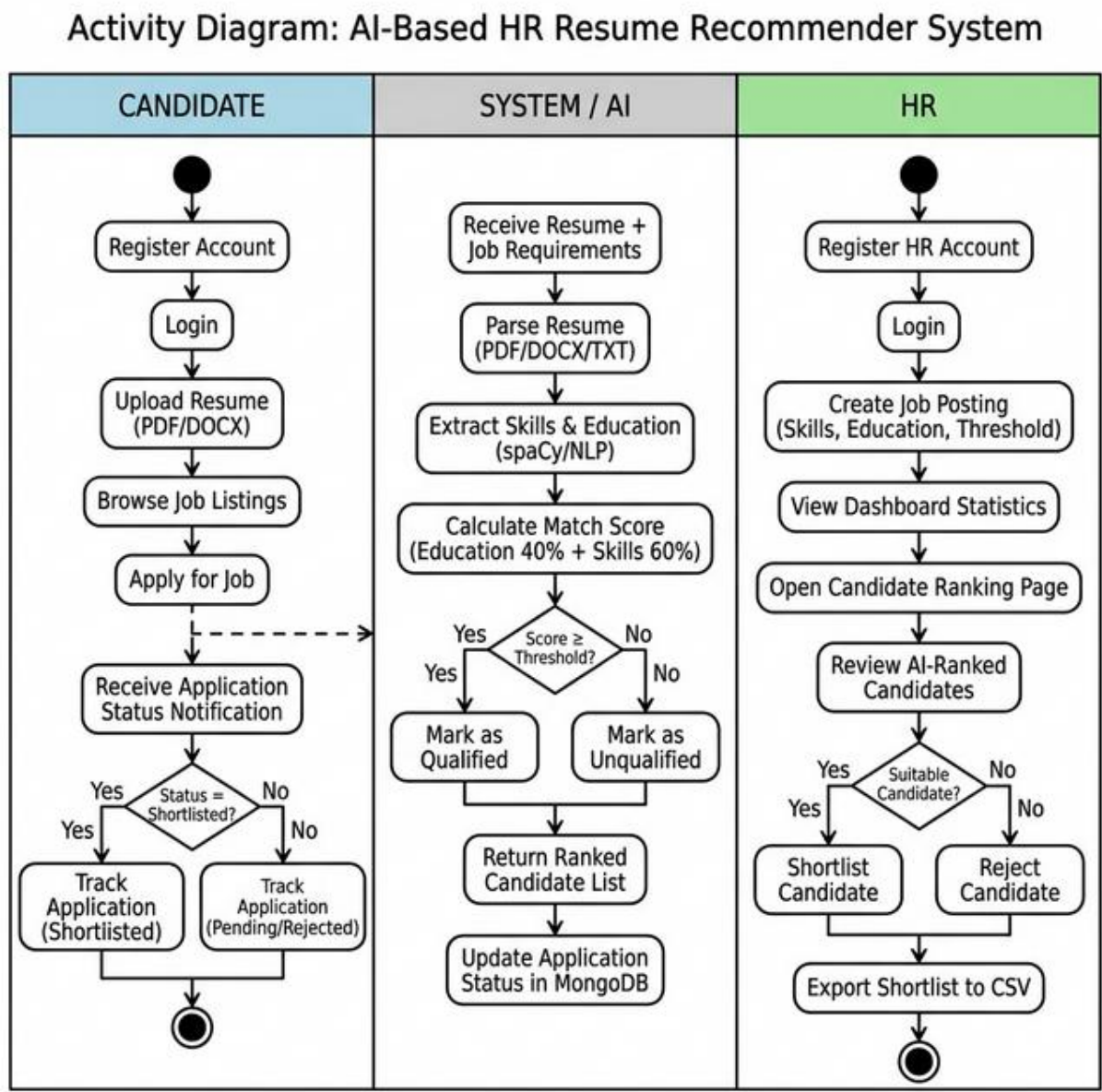


Figure 7.3: UML Activity Diagram — Full System Workflow with Swimlanes (Final Implementation)

7.1.3 Comparison with Initial Design

The following table documents the differences between the initial design (as proposed) and the final implemented system, with reasoning for each change:

Table 7.1: Comparison of Initial Design vs Final Implementation

Area	Initial Design	Final Implementation	Reason for Change
Application Statuses	Pending, Reviewed, Shortlisted, Rejected (4 statuses)	Pending, Shortlisted, Rejected visible to candidates; Reviewed retained internally only	UAT revealed "Reviewed" caused confusion for candidates — removed from candidate-facing view
AI Module Endpoint	Single /analyze endpoint	/api/analyze with expanded response (matched skills list, education score, skill score, overall score)	Additional data fields required to support the dashboard visualisations and skill breakdown display
Dashboard	Single summary table	Three summary cards + donut chart (Chart.js) + horizontal status bars	Stakeholder interview with Mrs Tracy confirmed need for visual, at-a-glance statistics
Manage Jobs Page	Not in original proposal	Fully implemented manage-jobs page with edit/delete, skill tags, and threshold setting	Identified as a necessary operational feature after Sprint 2 review
Resume Parser	PyPDF2 only	PyPDF2 → pdfminer fallback chain + python-docx	PyPDF2 failed on some PDF formats;

			fallback chain added for robustness
Architecture	Three-tier (as planned)	Three-tier (unchanged)	Architecture proved appropriate; no changes required

7.2 Database Structure

7.2.1 Final Database Design — MongoDB Collection Schema

The system uses MongoDB as its database, storing data across three primary collections. As a document-based NoSQL database, MongoDB does not enforce foreign key constraints at the database level; relationships between collections are managed through Mongoose ObjectId references at the application layer.

Figure 7.4 shows the MongoDB collection schema as implemented in the final system:

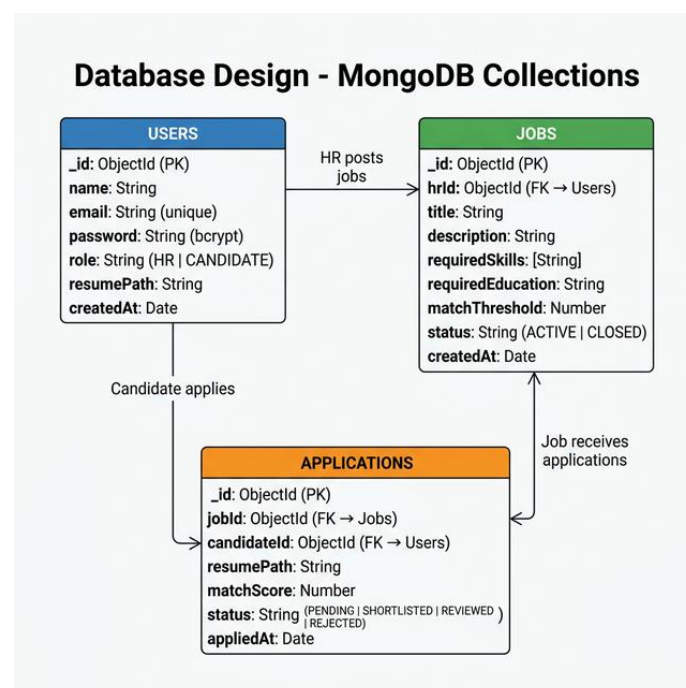


Figure 7.4: Final MongoDB Collection Schema — Users, Jobs, and Applications

The three collections and their key fields are:

Users Collection:

- `_id` (ObjectId), `name` (String), `email` (String, unique), `password` (String, bcrypt-hashed), `role` (Enum: 'HR' / 'CANDIDATE'), `skills` (Array<String>), `education` (String), `experience` (Number), `resumePath` (String), `createdAt` (Date)

Jobs Collection:

- `_id` (ObjectId), `title` (String), `description` (String), `requiredSkills` (Array<String>), `requiredEducation` (String), `minMatchScore` (Number, default: 0), `hrId` (ObjectId → Users), `createdAt` (Date)

Applications Collection:

- `_id` (ObjectId), `candidateId` (ObjectId → Users), `jobId` (ObjectId → Jobs), `resumePath` (String), `overallScore` (Number), `educationScore` (Number), `skillScore` (Number), `matchedSkills` (Array<String>), `status` (Enum: 'Pending' / 'Shortlisted' / 'Reviewed' / 'Rejected'), `appliedAt` (Date)

7.2.2 Design Rationale

Why MongoDB over SQL?

Schema flexibility suits variable resume and skill data; BSON maps directly to the JSON used throughout the stack; and Mongoose schemas provide sufficient application-layer validation without database-level schema management.

`resumePath` is stored in both Users and Applications intentionally — if a candidate updates their resume, historical application scores remain based on the original submission. `overallScore`, `educationScore`, `skillScore`, and `matchedSkills` are pre-computed and stored to avoid re-scoring on every request.

7.3 Completed Requirements / User Stories

7.3.1 MVP Features — Implemented

All Minimum Viable Product (MVP) features were successfully implemented and are fully functional in the submitted system:

Table 7.2: MVP Feature Completion Status

ID	User Story	Status
FR-H1	As an HR user, I want to register and log in so I can access the HR portal securely	 Complete
FR-H2	As an HR user, I want to post job vacancies with required skills and education, so candidates can apply	 Complete
FR-H3	As an HR user, I want to view AI-ranked candidates for each job, so I can identify the best matches quickly	 Complete
FR-H4	As an HR user, I want to shortlist or reject candidates, so I can record my decisions	 Complete
FR-C1	As a candidate, I want to register and log in so I can access the candidate portal	 Complete
FR-C2	As a candidate, I want to upload my resume, so it can be analysed by the AI	 Complete
FR-C3	As a candidate, I want to browse and apply for available jobs	 Complete
FR-C4	As a candidate, I want to view my application status and AI match score	 Complete
FR-S1	The system must extract skills and education from resumes using NLP	 Complete
FR-S2	The system must rank candidates by AI match score	 Complete
FR-S3	The system must only show qualified candidates (above threshold) to HR	 Complete

7.3.2 Post-MVP Features — Implemented

The following features were not in the original MVP definition but were implemented based on stakeholder feedback and project progress:

Table 7.3: Post-MVP Features Implemented

Feature	Description	Status
HR Dashboard Visualisations	Donut chart (Chart.js) showing Shortlisted vs Rejected; horizontal status bars for at-a-glance screening progress	 Implemented
Manage-Jobs Page	Centralised job management with edit/delete, skill tag display, and threshold configuration	 Implemented
Candidate Profile Management	Candidates can update their profile details and re-upload their resume	 Implemented
CSV Export	HR can export a shortlist of candidates for a job to a downloadable CSV file	 Implemented
Match Score Colour Coding	Visual colour indicators (green/amber/red) on candidate ranking table based on score range	 Implemented

7.3.3 Confirmation of Project Objectives

Table 7.4: Project Objectives — Completion Status

Objective	Achieved?	Evidence
Develop an AI module that parses resumes and extracts skills and education	Yes	AI module at /ai-module/ with spaCy pipeline and TF-IDF scoring
Rank candidates by AI match score against job requirements	Yes	GET /api/hr/jobs/:id/candidates returns sorted ranked list
Provide HR with a dashboard showing ranked candidates and clear match scores	Yes	candidates.html page with score badge, skill breakdown, and status controls
Implement role-based access with JWT authentication	Yes	protect + requireRole middleware on all protected routes
Provide candidates with real-time application status and match score visibility	Yes	applications.html page with score, status, and filter

7.4 User Testing Results

7.4.1 Description of User Testing Conducted

User Acceptance Testing (UAT) was conducted in Stage 4 (Weeks 15–18) using two dedicated test accounts (one HR, one Candidate) and a pre-seeded demo dataset (Junior Software Engineer posting with three candidate profiles). Each test was executed manually through the browser and results recorded. A seed script (setup-demo.js) ensured a consistent, reproducible test environment.

7.4.2 User Acceptance Testing Results

Table 7.5: User Acceptance Testing Results

#	Test Case	Role	Expected Outcome	Actual Outcome	Result
1	Register and log in	Candidate	JWT token issued; redirect to candidate portal	Token issued; portal loaded correctly	 Pass
2	Upload PDF resume	Candidate	Resume stored; confirmation shown	Resume stored at correct path; filename confirmed	 Pass
3	Browse and apply for job	Candidate	Application created; status set to Pending	Application saved; visible in HR ranking	 Pass
4	Post job with required skills	HR	Job saved and visible in manage-jobs page	Job appeared with correct skill tags	 Pass
5	View AI-ranked candidates	HR	Candidates ranked by score descending	Candidates ranked correctly; match scores accurate	 Pass
6	Update application to Shortlisted	HR	Status updated; reflected on candidate's page	Status updated in DB; visible to candidate immediately	 Pass
7	Export candidate list to CSV	HR	CSV file downloaded with name, score, status, skills	CSV downloaded with correct fields and data	 Pass

All 7 UAT test cases passed. No critical defects were found during UAT.

7.4.3 Key Usability Findings and Observations

Finding	Observation	Action Taken
Reviewed status confusion	Candidates could not distinguish between "Reviewed" and "Shortlisted" statuses	Removed "Reviewed" from candidate-facing filter; retained internally for HR workflow
Score readability	HR users found plain percentage numbers difficult to compare briefly across many candidates	Added colour-coded score badges (green $\geq 70\%$, amber 40–69%, red $< 40\%$) to ranking table
Ranking load speed	Initial implementation triggered AI re-scoring on every dashboard load	AI scoring moved to on-demand trigger (only when HR opens candidate ranking page for a specific job)
Resume upload feedback	Candidates were uncertain whether their resume had uploaded successfully	Added explicit success/error notification banner after upload

7.5 Final Interface Output Screenshots

This section provides visual evidence of the final implemented system for both HR and Candidate users. The interfaces confirm that core requirements were delivered and integrated successfully.

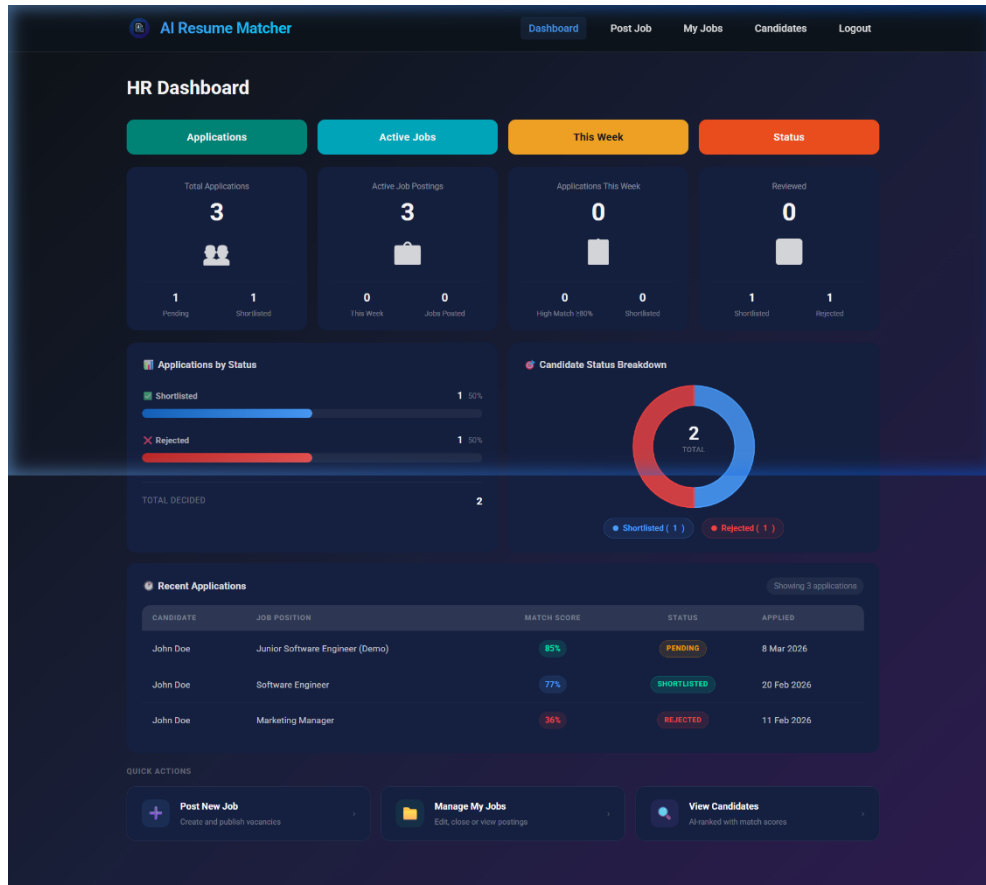


Figure 7.5: HR Dashboard — Summary Cards and Status Charts

Displays key recruitment metrics and visual charts to support quick monitoring and decision-making.

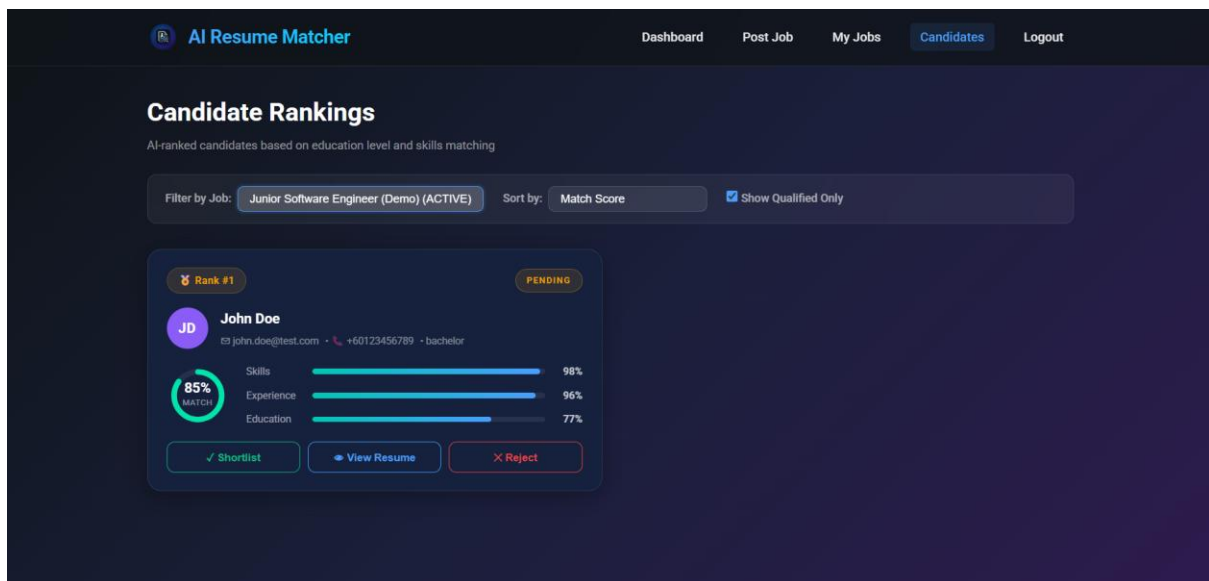


Figure 7.6: HR Candidate Ranking Page — AI-Ranked Candidates

Shows candidates ranked by AI match score, with matched skills, education indicators, and status controls for shortlisting or rejection.

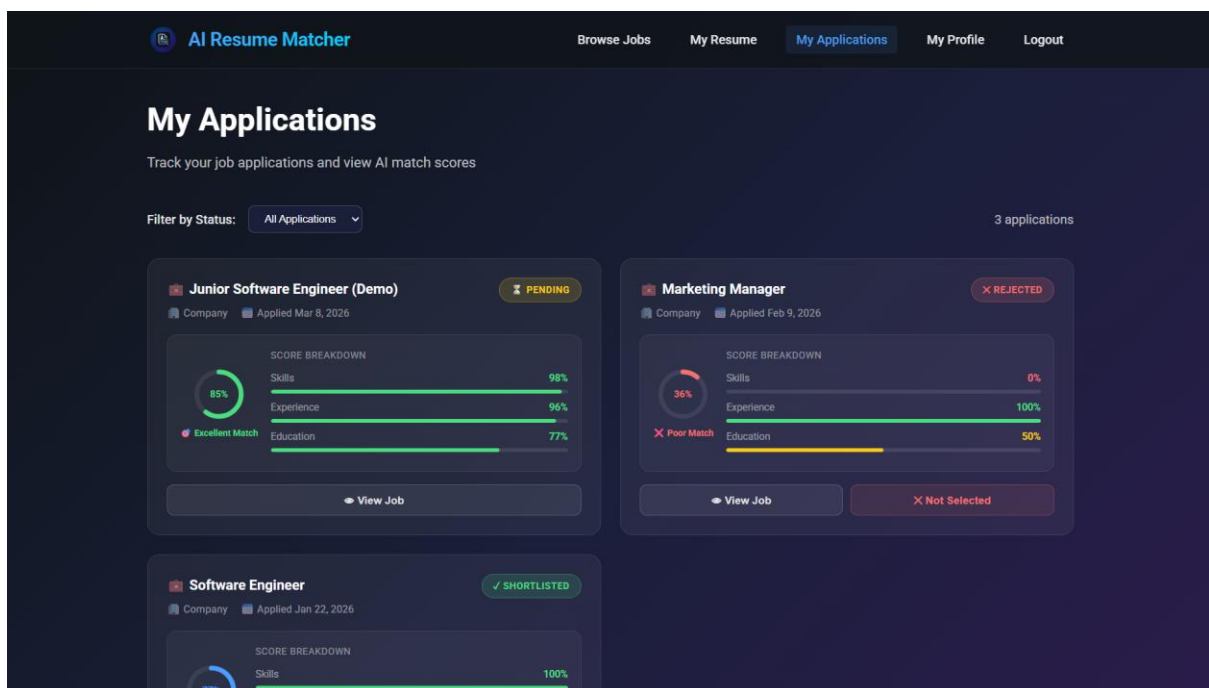


Figure 7.7: Candidate Portal — My Applications

Allows candidates to track applications with visible statuses and match scores for transparency.

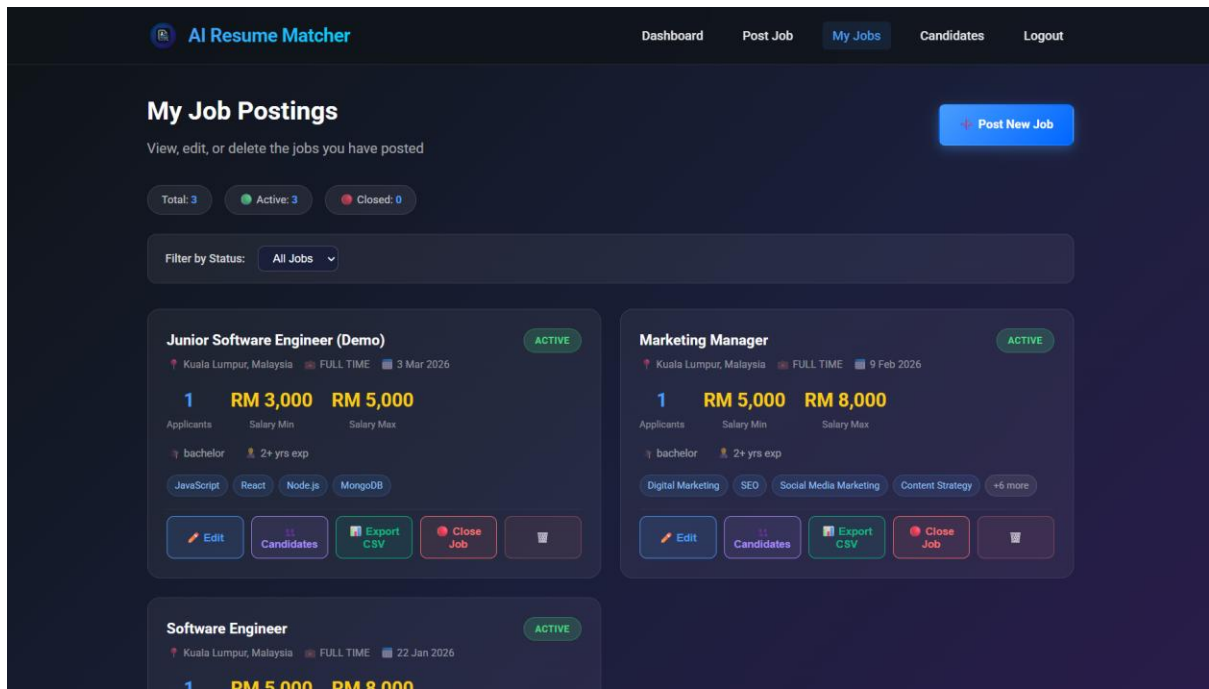


Figure 7.8: HR Manage Jobs Page

Enables HR users to create, edit, and delete job postings, including required skills and thresholds.

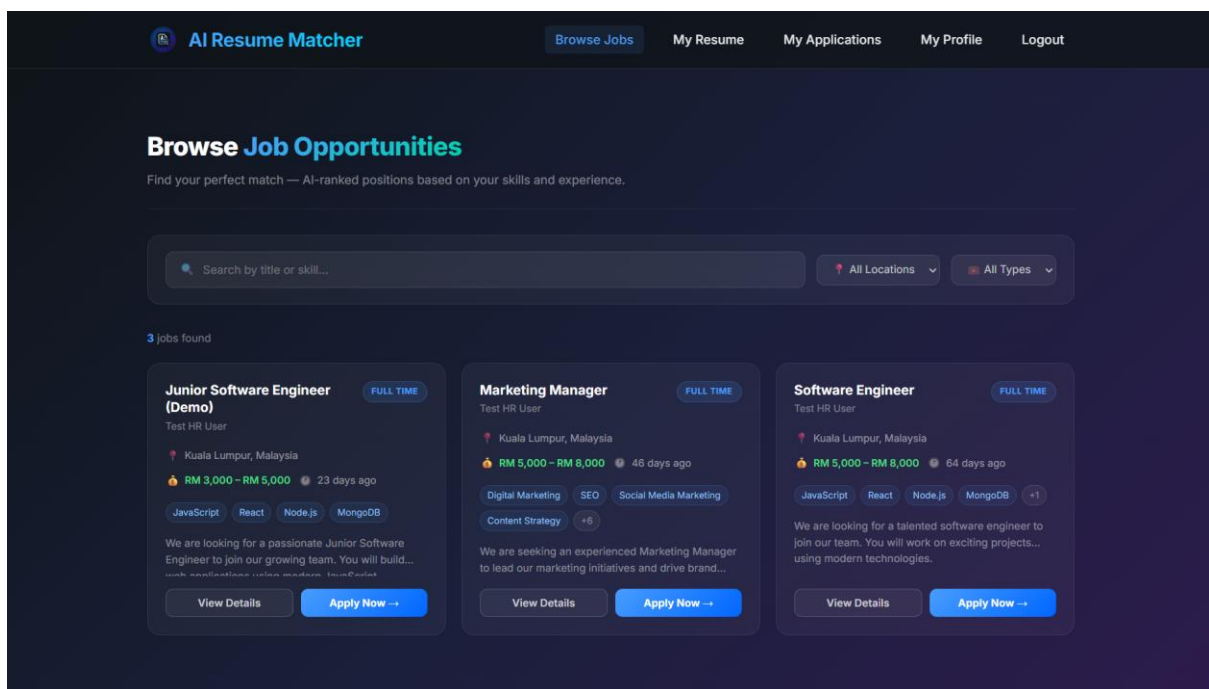


Figure 7.9: Candidate Job Listings and Apply Flow

Shows available vacancies and supports direct application submission using uploaded resumes.

The screenshot shows the Microsoft Excel interface with the 'Home' tab selected. The ribbon includes options for Clipboard, Font, Alignment, Number, and Styles. The spreadsheet data is as follows:

	A	B	C	D	E	F	G	H	I	J	K
1	Rank	Name	Email	Job	Overall Match	Skills Score	Experience Score	Education Score	Status	Applied Date	
2	1	John Doe	john.doe@test.com	Junior Software Engineer (Demo)	85%	98%	96%	77%	PENDING	8-Mar-26	
3											
4											
5											
6											
7											

Figure 7.10: CSV Export Output

Confirms shortlist export functionality for sharing candidate data with hiring stakeholders.

Overall, Figures 7.5 to 7.10 demonstrate an end-to-end workflow from posting jobs and applying to AI scoring, HR decision-making, and candidate status visibility.

Chapter 8: End-Project Report / Evaluation

This chapter provides a critical evaluation of the AI-Based HR Resume Recommender System against its original objectives and design. The purpose is not simply to summarise what was built, but to critically assess the quality, completeness, and effectiveness of the final system — acknowledging both its strengths and its limitations honestly.

8.1 Design Adherence

8.1.1 Proposed vs Final Implemented Design

The system was designed during Stage 1 as a three-tier microservice application: a JavaScript frontend served via a static server, a Node.js/Express.js backend, and a Python/FastAPI AI module. The final implemented system adheres to this core architecture without fundamental deviation.

Table 8.1: Design Adherence Summary

Design Area	Proposed Design	Final Implementation	Adherence
System Architecture	3-tier microservice (Frontend, Backend, AI Module)	3-tier microservice — unchanged	Full
Authentication	JWT-based, role-separated (HR / Candidate)	JWT with bcrypt, requireRole middleware	Full
AI Scoring	TF-IDF skill matching + education scoring	TF-IDF with spaCy PhraseMatcher + education classifier	Full
Database	MongoDB with Mongoose schemas	MongoDB with 3 collections (Users, Jobs, Applications)	Full
Resume Parser	PDF extraction	PyPDF2 + pdfminer fallback + python-docx	Enhanced

Candidate Ranking	Sorted ranked list visible to HR	Ranked table with score badges, skill breakdown, status controls	Enhanced
Dashboard Visuals	Simple table of statistics	Chart.js donut chart + horizontal status bars	Enhanced
Manage-Jobs Page	Not in original proposal	Fully implemented with edit/delete and threshold setting	Added
CSV Export	Not in original proposal	Implemented with correct field filtering	Added

8.1.2 Architecture Changes

No fundamental changes were made to the system architecture. The three-tier separation was maintained throughout development. The only structural addition was the introduction of aiService.js as a dedicated proxy module within the backend, which was not explicitly planned but improved the maintainability of the backend-AI integration by isolating all cross-service communication in one reusable module.

8.1.3 UI/UX Changes

The most notable UI/UX deviation from the initial design was the removal of the "Reviewed" status from the candidate-facing application portal. The original design included four statuses (Pending, Reviewed, Shortlisted, Rejected) on the candidate side, but UAT revealed this caused confusion. The "Reviewed" status was retained internally for HR workflow tracking but hidden from the candidate view, streamlining the candidate experience.

The addition of colour-coded score badges (green/amber/red based on score thresholds) was also not in the original specification but significantly improved HR usability, allowing recruiters to assess candidate quality immediately without reading individual percentage values.

8.1.4 Technology Changes

No technology changes were made from the proposed stack. All originally planned technologies — Node.js, Express.js, MongoDB, Python, FastAPI, spaCy, and TF-IDF — are present in the final system. The only addition was pdfminer as a fallback PDF parser alongside PyPDF2, which was not planned but became necessary after testing revealed PyPDF2's limitations with certain PDF encodings.

8.2 Objective Achievement

The five SMART objectives defined in Section 1.4 are revisited below, with an assessment of whether each was fully achieved, partially achieved, or not achieved, and justification for the assessment.

Table 8.2: Project Objectives — Achievement Status

#	Objective	Status	Justification
1	Develop an AI module that parses PDF/DOCX resumes and extracts skills and education level with measurable extraction accuracy	Fully Achieved	The AI module successfully parses PDF (via PyPDF2/pdfminer) and DOCX (via python-docx) resumes. Skill extraction uses spaCy PhraseMatcher against a pre-defined skill library. UAT confirmed correct extractions for all test resumes.
2	Implement a candidate ranking algorithm that ranks applicants by an AI match score with a documented, reproducible scoring formula	Fully Achieved	The scoring formula ($\text{Education} \times 0.4 + \text{Skills} \times 0.6$) is documented in Section 3.3 and is fully deterministic — identical input always produces identical output. Ranking is displayed correctly on the HR portal.
3	Build an HR portal that displays ranked candidates with match scores, skill breakdowns, and status management controls	Fully Achieved	The candidates.html page provides a ranked table with score badges, matched skills per candidate, status dropdowns (Shortlist/Reject), and a CSV export button. All UAT test cases for HR functionality passed.
4	Build a candidate portal that allows resume upload, job application, and real-time application status and match score visibility	Fully Achieved	The candidate portal includes profile management with resume upload, jobs.html for browsing and applying, and

			applications.html showing real-time status and AI match scores filtered by status. All candidate UAT test cases passed.
5	Deploy a complete, functional web application accessible via a local web server, documented with a comprehensive README	Fully Achieved	All three services run via npm run dev (backend), python -m uvicorn main:app (AI module), and VS Code Live Server (frontend). The README documents installation, configuration, and startup steps for all three services.

All five project objectives were fully achieved. No objectives were only partially met or unachieved.

8.3 System Evaluation

8.3.1 Strengths of the System

1. Transparent and interpretable AI scoring Unlike many enterprise ATS platforms that use opaque proprietary algorithms, this system's scoring is fully deterministic and documented. Both the formula ($\text{Education} \times 0.4, \text{Skills} \times 0.6$) and the component scores are visible to HR and candidates alike, addressing the "black box" concern raised in the literature (Bogen & Rieke, 2018).

2. Dual-portal architecture with role-based data isolation The strict separation between HR and Candidate portals — enforced at both the UI and API levels — means that candidates can only see their own data, and HR can only access applications for their own job postings. This role-based isolation is a significant security and privacy strength.

3. End-to-end workflow coverage The system covers the complete recruitment screening workflow: job posting → candidate application → AI scoring → ranked display → HR decision → candidate notification. This comprehensiveness distinguishes it from tools that only address a single step (e.g., resume parsing only, or job listing only).

4. Domain independence The skill library used by the AI module is not hardcoded to a specific industry. HR users specify required skills per job posting, and the AI matches against those — making the system applicable across sectors without reconfiguration.

5. Accessible and cost-free The system is built entirely on open-source technologies and can be deployed locally at no cost, making it accessible to SMEs and academic institutions that cannot afford enterprise ATS licensing.

8.3.2 Limitations and Weaknesses

1. English-only NLP model The spaCy model used (en_core_web_sm) only supports English-language text. Resumes submitted in Malay or other languages will not be correctly processed, limiting the system's applicability in Malaysia's multilingual workforce context.

2. Pattern-based skill extraction — no semantic understanding The skill matcher identifies skills by exact or near-exact string matching. A candidate who writes "data wrangling" when the job requires "data analysis" will receive no credit for that skill, even though the two terms refer to overlapping competencies. A semantic embedding approach (e.g., Sentence-BERT as suggested in Section 2.1.5.3) would address this but was beyond the project scope.

3. Image-based PDF resumes are not supported PDFs generated from scans or images (rather than digital text) cannot be parsed by PyPDF2 or pdfminer. These resumes return empty or garbled text, resulting in a 0% skill score. An OCR preprocessing step would be required to support this common resume format.

4. No HTTPS or production security hardening The current implementation runs over HTTP on localhost. There is no SSL/TLS encryption, no rate limiting, and no CSRF protection. The system is not suitable for public or production deployment in its current state without security hardening.

5. No automated testing suite All testing was conducted manually through the browser. The absence of unit tests (e.g., for the scoring algorithm) and integration tests (e.g., for API routes) means that future code changes carry a higher risk of introducing regressions without immediate detection.

6. No pagination on candidate lists For job postings with large numbers of applicants, the candidate ranking page loads all results at once. This could degrade performance with very large applicant pools.

8.3.3 Performance, Usability, and Reliability

Performance: AI scoring is triggered on demand when HR opens the candidate ranking page for a specific job. Response times observed during testing were under 2 seconds for typical resume files (1–2 pages, text-based PDFs), which is acceptable for a screening tool. Performance was not formally benchmarked but was qualitatively satisfactory across all UAT test cases.

Usability: All 7 UAT test cases passed on first attempt by the test user, suggesting that the core user flows are intuitive and correctly implemented. The addition of colour-coded score badges and the removal of the confusing "Reviewed" status further improved the candidate-facing experience during testing.

Reliability: The system performed stably across the full UAT session with no crashes or service failures. The PDF parser fallback chain (PyPDF2 → pdfminer) improved robustness for edge-case resume formats encountered during testing.

8.4 Future Enhancements

The following improvements are recommended for future versions of the system, prioritised by impact:

Table 8.3: Recommended Future Enhancements

Priority	Enhancement	Description
High	OCR for Image-Based PDFs	Integrate Tesseract OCR (via pytesseract) as a preprocessing step to extract text from scanned PDF resumes, expanding the range of supported resume formats
High	Automated Unit and Integration Tests	Implement a Jest test suite for backend API routes and a PyTest suite for the AI module's scoring functions, enabling regression-safe code changes
Medium	Semantic Skill Matching (Sentence-BERT)	Replace or augment the pattern-based skill matcher with a semantic similarity model (e.g., Sentence-BERT) to recognise skill synonyms and related competencies
Medium	Multi-Language NLP Support	Extend the NLP pipeline to support Bahasa Malaysia using a multilingual spaCy model or a language detection step, making the system suitable for the Malaysian market
Medium	Covering Letter / Self-Description Scoring	Allow candidates to submit a free-text covering letter alongside their resume, with the AI scoring relevance of the letter to the job requirements
Low	HTTPS and Production Security Hardening	Add SSL/TLS via a reverse proxy (e.g., Nginx), implement rate limiting on API endpoints, add CSRF tokens, and deploy to a cloud provider (e.g., Heroku, Render, or AWS EC2)
Low	Mobile-Responsive UI	Refactor the frontend CSS to be fully responsive on mobile and tablet devices, and ensure WCAG 2.1 accessibility compliance
Low	Commercial and Scalability Potential	The core architecture is compatible with cloud hosting (MongoDB Atlas, containerisation via Docker). With the above enhancements, the system could be offered as a SaaS product targeting Malaysian SMEs as a low-cost ATS alternative

Chapter 9: Post-mortem / Critical Reflection

This chapter provides a critical and honest post-mortem reflection on the project process, examining how the project was managed, what worked well, what did not, and what would be done differently with hindsight.

9.1 Technology Review

Table 9.1: Technology Evaluation — Strengths and Challenges

Technology	What Worked Well	What Caused Challenges	Would Choose Again?
Node.js / Express.js	Fast to set up, excellent middleware ecosystem, natural JSON handling	Async error handling sometimes obscured root causes	Yes
MongoDB / Mongoose	Schema flexibility suited evolving data structures	No foreign key enforcement: referential integrity managed in code	Yes
Python / FastAPI	Auto-generated Swagger UI invaluable; async support and fast startup	CORS configuration and Windows path issues with file references	Yes
spaCy (PhraseMatcher)	Deterministic, interpretable, fast for skill extraction	Limited to exact/near-exact matches; no semantic understanding	Yes (with caveats)
TF-IDF	Simple, fast, interpretable baseline; well-documented	Does not capture word relationships, limiting semantic accuracy	Partially
PyPDF2 / pdfminer	Covered most text-based PDF formats after fallback chain	PyPDF2 alone insufficient; encoding issues caused silent failures	Yes (with fallback)

Vanilla JavaScript	No build step; easy to debug in browser DevTools	Managing state across pages was verbose without a framework	Partially
Chart.js	Fast integration, visually polished charts	Minor bug when chart initialised before API data returned	Yes

With hindsight, two changes would be considered: **React.js** instead of Vanilla JavaScript for cleaner state management across pages, and **Sentence-BERT** alongside TF-IDF to recognise semantic skill equivalents (e.g., "machine learning" vs "ML").

9.2 Project Management Reflection

The Agile/Scrum methodology proved effective overall. The sprint structure provided clear short-term goals, and the MVP-first approach ensured core functionality was always working before enhancements were added. The GitHub Projects task board helped maintain visibility, though board discipline declined in later sprints when tasks were completed before being formally logged.

Two tasks were notably underestimated: **AI-backend integration** (CORS issues and Windows path differences added roughly one week of unplanned effort) and **PDF parser reliability** (the pdfminer fallback chain was not anticipated). Both were absorbed within sprint buffers without impacting the submission deadline.

Key lessons: define API contracts before writing implementation code; smoke-test cross-service communication from Sprint 1; and maintain the task board with daily discipline to preserve its value as a tracking tool.

9.3 Personal Performance

This project produced significant growth across multiple domains. Technically, the project was the first opportunity to build a complete three-tier web application, implement NLP techniques in a production-oriented context, and design a microservice architecture with real inter-service communication. Security implementation — JWT authentication, bcrypt hashing, and role-based access control — provided hands-on experience beyond textbook knowledge.

Soft skills developed include self-directed project management over 20 weeks, technical report writing at academic standard, and stakeholder communication through the PKT interview, which translated practitioner requirements into concrete design decisions.

Table 9.2: Challenges Faced and Resolutions

Challenge	How It Was Handled
AI-Backend integration (CORS + file path issues)	Systematic debugging via Postman and FastAPI Swagger UI; resolution through aiService.js abstraction
PDF parsing failures	Implemented try/except fallback chain (PyPDF2 → pdfminer)
State management across Vanilla JS pages	Used localStorage for JWT; query parameters for job selection
Balancing development with report writing	Enforced documentation-in-parallel discipline throughout all sprints

Areas for personal improvement: automated testing should be adopted from the start of future projects; UI should be wireframed before coding; and design decisions should be logged in real-time rather than reconstructed from memory.

9.4 User Testing Reflection

The UAT covered 7 test cases using two test accounts and a pre-seeded demo dataset. All 7 passed. However, testing was limited to a single HR and Candidate user, used only well-formatted text-based PDFs, and included no load testing — all acknowledged limitations given the solo academic scope. The UAT was conducted internally by the developer, introducing an inherent bias, as the developer knows the system's intended behaviour.

Despite these constraints, testing produced two concrete improvements: the **"Reviewed" status was removed from the candidate-facing view** (identified as confusing), and **match score colour coding was added** (identified as a readability improvement). These changes demonstrate that even limited user testing produces actionable feedback when conducted with openness to change.

9.5 Ethical and Accessibility Reflection

The system does not currently meet **WCAG 2.1 Level AA** standards — ARIA labels are absent from interactive elements, and colour-coded score badges rely solely on colour to convey meaning, which is inaccessible to users with colour blindness. No keyboard-only navigation testing was conducted.

Ethically, three gaps remain: no explicit data consent form at registration (a gap relative to PDPA 2010 and GDPR), potential skill library bias disadvantaging candidates from non-mainstream backgrounds, and no right-to-erasure mechanism for candidate data. Future versions should address all three alongside WCAG compliance improvements.

9.6 Overall Reflection

The AI-Based HR Resume Recommender System is honestly assessed as a **successful project within its defined scope** — all five objectives were fully achieved, all MVP features delivered, and the system functions as a coherent, working end-to-end application. However, it is a functional proof-of-concept, not a production-ready product; HTTPS, automated testing, OCR, and accessibility compliance remain to be addressed before commercial deployment.

The most valuable outcome was not the system itself, but the transferable skills built through the process: multi-tier architecture design, AI/NLP integration, solo project management, and academic documentation. Four key takeaways will inform future practice: interpretability matters in AI systems; integrations must be tested early; good requirements reduce rework; and documentation should be maintained in parallel with development.

Chapter 10: Conclusion

10.1 Project Summary

10.1.1 Recap of the Problem Addressed

Manual resume screening in SME recruitment is time-consuming, inconsistent, and offers candidates no feedback on their application outcomes. Existing solutions — enterprise ATS platforms and commercial job portals — are either too costly for SMEs, too rigid in their keyword-matching logic, or opaque in how they rank candidates.

This project addressed five specific, documented problems: excessively manual screening workflows, inconsistent candidate evaluation, a lack of transparency for candidates, difficulty managing multiple job postings, and the absence of a structured shortlisting workflow.

10.1.2 Summary of the Developed Solution

The AI-Based HR Resume Recommender System automates the resume screening and candidate ranking stage of recruitment across three interconnected tiers: a frontend with separate HR and Candidate portals; a Node.js/Express.js backend managing authentication, data, and AI communication; and a Python/FastAPI AI module that parses resumes, extracts skills and education using spaCy, and computes a weighted match score ($\text{Education} \times 0.4 + \text{Skills} \times 0.6$) using TF-IDF. Match scores and application statuses are displayed in real-time to both HR and candidates.

10.2 Achievement of Aim and Objectives

10.2.1 Confirmation of How the Project Aim Was Met

The overarching aim of the project was:

"To design and develop an AI-powered web-based system that automates the resume screening and candidate ranking process for HR professionals, while providing transparent match feedback to candidates."

This aim has been fully achieved. The system automates resume screening (via the AI module's parsing and scoring pipeline), ranks candidates (via the scoring algorithm and ranked display on the HR portal), and provides transparent feedback to candidates (via visible match scores and real-time application status on the candidate portal).

10.2.2 High-Level Summary of Objective Completion

Objective	Outcome	
Develop an AI module to parse resumes and extract skills and education	✓	Fully Achieved
Implement a reproducible candidate ranking algorithm with documented formula	✓	Fully Achieved
Build an HR portal with ranked candidates, match scores, and status management	✓	Fully Achieved
Build a candidate portal with resume upload, job application, and status visibility	✓	Fully Achieved
Deploy a complete, functional web application with a comprehensive README	✓	Fully Achieved

All five objectives were fully achieved. No objectives were partially met or unachieved.

10.3 Contribution of the Project

10.3.1 Academic Contribution

This project demonstrates the practical application of established NLP and information retrieval techniques — TF-IDF (Salton & Buckley, 1988) and spaCy pattern matching — to a real-world recruitment problem. It provides a working reference implementation of a resume scoring system that is interpretable, deterministic, and accessible, addressing the transparency gap identified in the literature (Bogen & Rieke, 2018; Raghavan et al., 2020). The system also demonstrates how a Python-based AI module can be cleanly integrated with a Node.js web backend as separate microservices, a pattern applicable to a broad range of AI-augmented web application scenarios.

10.3.2 Practical and Industry Relevance

The system addresses a genuine need in the Malaysian SME recruitment landscape. As confirmed through the stakeholder interview with PKT's HR Executive (Section 2.1.5), organisations without enterprise ATS budgets rely on entirely manual processes that are both inefficient and inconsistent. This project demonstrates that effective AI-assisted screening is achievable with entirely open-source technologies, at zero licensing cost, and at a level of sophistication comparable to features found in premium recruitment platforms.

10.3.3 Learning Outcomes Achieved

Through this project, the following learning outcomes were achieved:

- **Full-stack web development** — Design and implementation of a complete three-tier web application from authentication to data visualisation
- **AI/NLP engineering** — Practical application of NLP techniques in a production-oriented microservice context
- **Software engineering practices** — Agile methodology, sprint planning, version control, structured testing, and technical documentation
- **Professional and ethical awareness** — Understanding of data protection legislation (PDPA 2010, GDPR), algorithmic fairness principles, and professional codes of conduct (ACM, BCS) in the context of an AI system that affects hiring outcomes

10.4 Final Remarks

10.4.1 Overall Success Statement

The AI-Based HR Resume Recommender System is a complete, functional, and well-documented project that successfully meets all of its defined objectives. It demonstrates that a solo developer, working within the scope of a final-year academic project, can design and deliver a meaningful AI-augmented web application that addresses a real-world problem with practical industry relevance.

10.4.2 Readiness for Real-World Deployment

The system is currently ready for **demonstration and prototype deployment** in a controlled environment. Before broader public or commercial deployment, HTTPS, rate limiting, OCR support, a PDPA-compliant consent mechanism, and a formal security audit would need to be addressed. The architecture is cloud-compatible and could be deployed to MongoDB Atlas and a Python API hosting service with modest additional engineering effort.

10.4.3 Closing Reflection

This project has been the most comprehensive and technically demanding individual undertaking of the degree programme. It required the simultaneous application of knowledge from software engineering, artificial intelligence, web development, database design, project management, and professional ethics — not in isolation but integrated into a single coherent system.

The process reinforced a conviction that became clearer across each sprint: good engineering is not just about writing code that works, but about making deliberate, documented, and justifiable decisions at every level — from architecture to algorithm design to user experience. The AI-Based HR Resume Recommender System is not just a working application; it is evidence of a development process that was thoughtful, structured, and genuinely informed by the needs of real users.

References

- Bogen, M., & Rieke, A. (2018). *Help wanted: An examination of hiring algorithms, equity, and bias*. Upturn. <https://www.upturn.org/reports/2018/hiring-algorithms/>
- Cappelli, P. (2019). Your approach to hiring is all wrong. *Harvard Business Review*, 97(3), 48–58. <https://hbr.org/2019/05/your-approach-to-hiring-is-all-wrong>
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Ferreira, R., Lima, D., Rodrigues, A., & Santos, P. (2022). Comparing rule-based and deep learning approaches for resume parsing in human resources systems. *International Journal of Information Management Data Insights*, 2(1), 100061. <https://doi.org/10.1016/j.jjimei.2022.100061>
- Harsha, M., Preethi, G., & Rajan, K. (2020). Resume parser: A survey on information extraction from resumes. *International Journal of Advanced Research in Computer Science*, 11(3), 14–19.
- JobStreet Malaysia. (2025). *Recruitment insight report: Malaysia talent landscape*. JobStreet by SEEK. <https://www.jobstreet.com.my>
- Liao, X., Jiang, Z., & Kwan, P. (2018). A novel approach to resume entity recognition using deep learning. *Proceedings of the 2018 International Conference on Computational Intelligence and Data Engineering*, 113–120. https://doi.org/10.1007/978-981-10-8228-3_11
- Lindsey, A., King, E., Hebl, M., & Levine, N. (2013). The impact of method, motivation, and empathy on diversity training effectiveness. *Journal of Business and Psychology*, 30(3), 605–617. <https://doi.org/10.1007/s10869-014-9384-3>
- LinkedIn Talent Solutions. (2025). *Global talent trends report 2025*. LinkedIn Corporation. <https://www.linkedin.com/talent/>
- Maheshwari, S., Sainani, A., & Reddy, P. K. (2010). An approach to extract special skills for resume information extraction. *Proceedings of the IEEE International Symposium on Database Technology and Applications*, 1–4. <https://doi.org/10.1109/DBTA.2010.5658955>
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26, 3111–3119. <https://dl.acm.org/doi/10.5555/2999792.2999959>

- Qin, C., Zhu, H., Xu, T., Zhu, C., Ma, L., Chen, E., & Xiong, H. (2018). Enhancing person-job fit for talent recruitment: An ability-aware neural network approach. *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, 25–34. <https://doi.org/10.1145/3209978.3210025>
- Raghavan, M., Barocas, S., Kleinberg, J., & Levy, K. (2020). Mitigating bias in algorithmic hiring: Evaluating claims and practices. *Proceedings of the 2020 ACM Conference on Fairness, Accountability, and Transparency*, 469–481. <https://doi.org/10.1145/3351095.3372828>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing and Management*, 24(5), 513–523. [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0)
- Upadhyay, A. K., & Khandelwal, K. (2018). Applying artificial intelligence: Implications for recruitment. *Strategic HR Review*, 17(5), 255–258. <https://doi.org/10.1108/SHR-07-2018-0051>
- Zhang, J., Zhao, Y., Saleh, M., & Liu, P. J. (2016). *ROUGE 2.0: Updated and improved measures for evaluation of summarization tasks*. arXiv:1803.01937. (Applied in the context of text similarity evaluation for job-candidate matching.)

Appendix

Appendix A — User Interview Transcript (PKT HR Stakeholder Interview)

This appendix contains the full set of interview questions and recorded answers obtained from the stakeholder interview with **Mrs Tracy (PKT HR Executive)**. The interview provided primary research evidence used to shape requirements and key design decisions (refer to Section 2.1.5).

A. Job Posting Process

1. How does HR normally create and post job vacancies?

HR uses platforms like Indeed, JobStreet (for hard-to-fill roles), Facebook company page, and MyFutureJobs. Free platforms are preferred due to zero hiring cost policy.

2. What information do you include in a job posting (e.g., title, description, skills, salary range)?

Title, job description, required skills, salary range (in RM), benefits, allowances, and expected start date.

3. Do you use any existing platforms or internal systems for posting jobs?

Yes, Indeed and Facebook are commonly used, occasionally JobStreet and MyFutureJobs.

B. Resume Submission

4. How do candidates currently submit their resumes?

Mostly through Indeed portal, sometimes via email or WhatsApp.

5. Do you receive resumes through email, portal, or manually?

Yes, all three methods are used, but portal submission is most common.

6. Are there preferred file formats (PDF, DOCX)?

PDF is preferred.

C. Screening Process

7. How do you shortlist candidates?

First check education level, then location, then work experience.

8. Do you check education first, skills, or experience?

Education first, then experience if required by the position.

9. What are the main criteria you look for when screening resumes?

Education qualification, relevant skills, location, and experience.

10. Would you like to set custom weightage for education, skills, and experience?

Yes, HR would like flexibility to adjust weightage.

D. Selection Flow

11. After shortlisting, what is the next step?

HR sends shortlisted resumes to Hiring Manager, then Department Head, then HQ for final approval.

12. Who approves the selected candidates?

Hiring Manager and Department Head, followed by HQ.

13. Is there a standard format or evaluation template used?

Currently, no formal template; decisions are based on resume review and background checks.

E. Pain Points

14. What are the challenges with the current recruitment process?

Hard to find skilled employees, long approval process, manual filtering is time-consuming.

15. Which tasks take the most time or effort?

Approval process (up to one week) and manual resume screening.

16. What would HR like to automate in the process?

Resume filtering and ranking using AI.

F. System Expectations

17. What features do you expect from this new system?

Auto-ranking, dashboard with candidate details (name, phone, address, qualifications, job history), and ability to export lists.

18. Do you want the system to generate ranked candidates automatically?

Yes.

19. What information should appear in the HR dashboard (e.g., candidate score, skills matched, resume preview)?

Candidate score, skills matched, education level, resume preview, contact details.

20. Should the system integrate with existing HR tools (e.g., JobStreet, LinkedIn)?

Yes, integration with Indeed and JobStreet would be helpful.

21. Are there any privacy or compliance requirements for storing resumes?

No strict internal policy, but best practice is to secure data.

G. Future Enhancements

22. Would you like features such as skill gap analysis or predictive hiring success in the future?

Yes.

23. Should the system support interview scheduling or candidate communication later?

Yes, like Indeed's scheduling feature.

Appendix B — API Endpoint Reference

Tables B.1 to B.5 summarise the REST API endpoints implemented at project submission. The **Node.js/Express** backend (default base URL `http://localhost:5000`) handles authentication, HR and candidate workflows, file uploads, and CSV export. The **Python FastAPI** AI microservice (default base URL `http://localhost:8000`) handles resume parsing, single-resume analysis, and batch candidate ranking. The backend calls the AI service over HTTP as described in Chapter 3.

Authentication. Protected backend routes require a JSON Web Token: send header `Authorization: Bearer <token>`. Tokens are returned by `POST /api/auth/register` and `POST /api/auth/login`. HR routes require role `HR`; candidate routes require role `CANDIDATE`, except where noted as public.

JSON convention. Successful responses typically use `{success: true, ...}` with a data or message field. Errors use HTTP status codes (400, 401, 403, 404, 500) with `{success: false, message: “...”}`.

Table B.1 — Server utility endpoints (Express)

Method	Path	Auth	Description	Typical Success Response
GET	/health	None	Health check	JSON: status, message, timestamp
GET	/	None	API information	JSON: message, version, endpoints

Table B.2 — Authentication (/api/auth)

Method	Path	Auth	Request Body	Typical Success Response
POST	/api/auth/register	None	email, password, role, fullName, phone; optional education, skills	201 — data.user, data.token
POST	/api/auth/login	None	email, password	200 — data.user, data.token
GET	/api/auth/me	JWT	—	200 — data.user

Table B.3 — HR portal (/api/hr)

All routes require **JWT** and role **HR**.

Method	Path	Request Body / Parameters	Typical Success Response
POST	/api/hr/jobs	Job fields in JSON body; hrId set server-side	201 — data.job
GET	/api/hr/jobs	—	200 — data.jobs (includes applicantCount)
GET	/api/hr/jobs/:id	Path: job ID	200 — data.job
PUT	/api/hr/jobs/:id	JSON fields to update	200 — data.job
DELETE	/api/hr/jobs/:id	Path: job ID	200 — confirmation message
GET	/api/hr/jobs/:id/candidates	Path: job ID; optional query params	200 — data.job, data.applications
GET	/api/hr/jobs/:id/export-csv	Path: job ID	200 — CSV file (UTF-8 BOM)
GET	/api/hr/dashboard	—	200 — dashboard statistics
PATCH	/api/hr/applications/:id/status	Body: { "status" }	200 — data.application

Table B.4 — Candidate portal (/api/candidates)

Method	Path	Auth	Request Body / Parameters	Typical Success Response
GET	/api/candidates/jobs	None	Optional query: search, location, employmentType	200 — data.jobs
GET	/api/candidates/jobs/:id	None	Path: job ID	200 — data.job
POST	/api/candidates/resume	JWT + CANDIDATE	multipart/form-data, field name resume	200 — data.resumePath, data.filename
POST	/api/candidates/apply	JWT + CANDIDATE	JSON: { "jobId" }	201 — data.application (includes matchScore, rankingDetails)
GET	/api/candidates/applications	JWT + CANDIDATE	—	200 — data.applications
GET	/api/candidates/profile	JWT + CANDIDATE	—	200 — candidate profile data
PUT	/api/candidates/profile	JWT + CANDIDATE	JSON: profile fields to update	200 — updated profile data

Table B.5 — AI module (FastAPI, base URL <http://localhost:8000>)

Used by the backend and for direct testing (interactive docs at /docs). Not on the same port as Express.

Method	Path	Auth	Request	Typical Success Response
GET	/	None	—	status, message, version
GET	/health	None	—	status, message, version
POST	/api/parse-resume	None	Multipart file upload (PDF/DOCX)	success, data (parsed analysis)
POST	/api/analyze	None	JSON: resume_path, job_data	success, resume_analysis, match_score
POST	/api/rank-candidates	None	JSON: resume_paths (array), job_data	success, ranked_candidates, total_candidates

The backend integrates with the AI layer primarily through **POST /api/rank-candidates** (batch ranking) and through the internal service that calls the AI during **POST /api/candidates/apply** for per-application scoring. Detailed scoring logic (skill match, experience, TF-IDF text similarity, education, and weighted overall score) is implemented in the AI module's matcher component and is described conceptually in Chapter 3.

Appendix C — Skill Library

This appendix documents the predefined skill vocabulary used by the AI module when extracting skills from resume text. The lists are defined in `ai-module/analyzers/skill_extractor.py` as **TECHNICAL_SKILLS** and **SOFT_SKILLS** and combined as **ALL_SKILLS**.

The function `extract_skills()` searches resume text using case-insensitive pattern matching with word boundaries, so only phrases that appear in the resume and match this vocabulary are recorded as candidate skills.

HR-defined required skills for each job are stored on the job posting. The skill match score compares those required skills to the skills extracted from the resume (see Chapter 3). Phrases not in this library are not detected automatically unless the code is extended or additional terms are passed via the optional `custom_skills` parameter.

Table C.1 — Technical Skills (by Category)

Category	Skill Phrases (alphabetical within category)
Programming Languages	bash, c, c#, c++, css, go, golang, html, java, javascript, kotlin, matlab, perl, php, python, r, ruby, rust, scala, shell, sql, swift, typescript
Web Frameworks	angular, asp.net, django, express, express.js, fastapi, flask, laravel, next.js, node.js, nodejs, nuxt.js, react, ruby on rails, spring, spring boot, svelte, vue, vue.js
Mobile Development	android, flutter, ionic, ios, react native, xamarin
Databases	cassandra, couchdb, dynamodb, elasticsearch, mariadb, mongodb, ms sql, mysql, neo4j, oracle, postgresql, redis, sqlite
Cloud & DevOps	ansible, aws, azure, ci/cd, devops, docker, gcp, github actions, gitlab, google cloud, jenkins, k8s, kubernetes, terraform, vagrant
Data Science & AI	ai, artificial intelligence, computer vision, data analysis, data science, deep learning, keras, machine learning, neural networks, nlp, numpy, pandas, pytorch, scikit-learn, tensorflow, transformers
Tools & Technologies	bitbucket, confluence, docker, git, github, gitlab, intellij, jira, kubernetes, linux, pycharm, unix, visual studio, vs code, windows
Methodologies	agile, graphql, kanban, microservices, rest api, scrum, soap, tdd, test-driven development, waterfall
Other Technical	api, authentication, authorization, grpc, integration testing, jwt, oauth, qa, rest, testing, unit testing, websocket

Table C.2 — Soft Skills

#	Skill Phrase
1	adaptability
2	analytical
3	attention to detail
4	collaboration
5	communication
6	conflict resolution
7	creativity
8	critical thinking
9	customer service
10	decision making
11	flexibility
12	interpersonal
13	leadership
14	multitasking
15	negotiation
16	organization
17	presentation
18	problem solving
19	project management
20	public speaking
21	self-motivated
22	team player

23	teamwork
24	technical writing
25	time management

Summary

Set	Count
Technical skills (TECHNICAL_SKILLS)	126
Soft skills (SOFT_SKILLS)	25
Combined (ALL_SKILLS)	151